Handelshögskolan
Karlstad Business School

Melker Christiansson

A Comparative Study of RBAC and ABAC in Microsoft Entra

Differences in Authorization Granularity, Administration Complexity, and Policy Adaptability

Information Systems

C-thesis

Term: Spring 2026
Supervisor: Peter Bellström

Karlstad Business School
Karlstad University SE 651 88 Karlstad
Phone: +46 54 700 10 00
E-mail: handels@kau.se kau.se/en/hhk

Abstract

Identity and Access Management (IAM) systems are critical components in modern cloud environments. Organizations must balance security, flexibility and administrative complexity when controlling access to resources. Within Microsoft Entra, Role-Based Access Control (RBAC) remains the dominant model, while Attribute-Based Access Control (ABAC) is increasingly discussed as a more flexible alternative. Despite extensive theoretical research on both models, there is limited empirical evidence comparing their practical behavior and implementation within this specific platform context.

This study addresses that gap by conducting an empirical comparison of RBAC and ABAC within a Microsoft Entra-based environment. A Proof-of-Concept (PoC) application was designed and implemented, where both authorization models were applied under identical conditions. Four controlled test scenarios were developed to isolate key characteristics, including resource filtering, sensitivity constraints, ownership-based access and write operations. Authorization behavior was evaluated using measurable indicators such as access decision, visible resources and evaluation scope, enabling systematic comparison between the models.

The results show that ABAC provides higher granularity and flexibility by enabling attribute-based, per-resource authorization decisions and supporting multi-attribute, context-dependent policies. In contrast, RBAC produces consistent and predictable access decisions based on role membership but requires additional roles to represent complex or dynamic access requirements. The study further shows that RBAC has lower initial implementation complexity, while ABAC introduces greater upfront complexity through policy design and attribute management. However, ABAC demonstrates improved adaptability, as access decisions automatically reflect changes in resource attributes, whereas RBAC requires manual updates to role assignments.

The findings indicate that neither model is universally superior. RBAC is well suited for stable environments with clearly defined roles, while ABAC is more appropriate for dynamic and context-dependent scenarios. The study contributes empirical evidence to the problem domain and highlights the practical trade-offs between role-based and attribute-based authorization in Microsoft Entra.

Table of contents

Abstract....2
Table of contents....3
List of tables....7
List of figures....1
1. Introduction....1
1.1 Purpose....2
1.2 Research questions....2
1.3 Scope and Delimitations....2
1.4 Ethical considerations....2
1.4.1 GenAI Disclaimer....3
2. Literature overview....4
2.1 Identity and Access Management in Cloud Environments....5
2.2 Fundamental Concepts in Access Control....6
2.2.1 Core Access Control concepts....6
2.2.2 Authorization granularity and flexibility....6
2.2.3 Administrative complexity and operational management....7
2.2.4 Implementation challenges, best practices and standards....8
2.3 Role-Based Access Control (RBAC)....9
2.3.1 Theoretical Foundations....9
2.3.2 Administrative Strengths and Limitations....10
2.3.3 RBAC Implementation in Microsoft Entra....10
2.3.4 Operation and Architectural Considerations for RBAC....11
2.4 Attribute-Based Access Control (ABAC)....11
2.4.1 Theoretical Foundations....11
2.4.2 Administrative Strengths and Challenges....12
2.4.3 Attribute-Based Controls in Microsoft Entra....13
2.4.4 Operation and Architectural Considerations for ABAC....13
2.5 Hybrid RBAC-ABAC Approaches....14
2.6 Microsoft Entra as an IAM platform....14
2.7 Implementation specific frameworks and technologies....15
2.7.1 Containerization with Docker....15
2.7.2 ASP.NET Core....15
2.8 Synthesis and Research Gap....16
2.9 Critical Review of the Literature....17
3. Method....19

3.1 Introduction to the Methodology 19
3.2 Research design 22
3.3 Implementation (PoC architecture) 23
3.3.1 System overview 23
3.3.2 RBAC Implementation 23
3.3.3 ABAC Implementation 24
3.3.4 MeasurementService 25
3.3.5 Test Scenarios 27
3.4 Data Collection Methods 28
3.4.1 Automated testing 28
3.4.2 Quantitative Measurement Collection 28
3.5 Data Analysis Methods 29
3.5.1 Empirical Data Analysis 29
3.5.2 Structured Comparative Analysis 30
3.5.3 Design Science Evaluation 30
3.6 Validity and Reliability 30
3.7 Limitations 31
3.8 Research Contributions 31
4. Implementation 32
4.1 System Overview 32
4.1.1 Project Attributes 32
4.1.2 User Attributes 33
4.1.3 Attribute design rationale 34
4.1.4 ABAC Authorization Logic 34
4.1.5 RBAC Authorization Logic 35
4.2 Test Scenarios 36
4.2.1 Scenario1 - Read-heavy project listing with filtered visibility 37
4.2.2 Scenario2 - Sensitive resource access control 38
4.2.3 Scenario3 - Ownership-based access overrides 39
4.2.4 Scenario4 - Write operations (create, update, delete) 40
4.3 Validation of Authorization Implementation 42
4.3.1 ABAC Validation 42
4.3.2 RBAC Validation 42
4.3.3 Validation Results Conclusion 43
5. Results 44
5.1 Scenario1 44
5.2 Scenario2 45

5.3 Scenario3....46
5.4 Scenario4....47
6. Analysis....48
6.1 RQ1 - Authorization Granularity and Flexibility....48
6.1.1 Per-resource vs role-based decision semantics....48
6.1.2 Conjunctive policy evaluation....48
6.1.3 Context-dependent and exception-based authorization....49
6.1.4 Conclusions for RQ1....49
6.2 RQ2 - Implementation and Administrative Complexity....49
6.2.1 Implementation complexity in practice....50
6.2.2 Structural vs policy-based complexity....50
6.2.3 Conclusion for RQ2....50
6.3 RQ3 - Policy Maintenance and Adaptability....50
6.3.1 Dynamic adaptation to attribute changes....51
6.3.2 Administrative overhead over time....51
6.3.3 Conclusion for RQ3....51
6.4 Overall Synthesis....52
7. Conclusion....53
7.1 Summary of findings....53
7.2 Answer to Research Questions....53
7.2.1 RQ1 - What are the practical differences between RBAC and ABAC in Microsoft Entra with respect to authorization granularity and flexibility?....53
7.2.2 RQ2 - How do RBAC and ABAC differ in terms of implementation complexity and administrative workflow within Microsoft Entra?....53
7.2.3 RQ3 - How do policy maintenance and change management differ when using RBAC vs ABAC in Entra?....54
7.3 Overall Conclusions....54
7.4 Contributions....55
7.5 Limitations....55
7.6 Future Work....56
8. Acknowledgements....57
Bibliography....58
Annex....60
Appendix A. DSR Guidelines Hevner et al., (2004)....60
Appendix B. Activities for conducting DSR Peffers et al., (2007)....61
Appendix C. Validation Test Results....62
Appendix D. Scenario Test Results....64
Appendix E. Implementation Log....67

Development Approach..... 67
System Architecture Overview..... 67
Backend Architecture and Routing..... 67
Persistence Layer and Data Management..... 68
Authentication Architecture (Microsoft Entra)..... 68
Authorization Architecture..... 68
Measurement and Instrumentation Design..... 69
Measurement Model Design..... 69
Reproducibility and Experimental Control..... 69
References Appendix E..... 71

List of tables

Table 1. List of Acronyms and Concepts..... 4

Table 2. How Peffers et al., (2007, pp. 52, 55-56) Activities for conducting DSR was followed in this thesis..... 20

Table 3. How Hevner et al., (2004, p. 83) guidelines for DSR were followed in this thesis..... 21

Table 4. Measurement Variables collected from the artefact..... 26

Table 5. Test-scenarios used for automated tests..... 27

Table 6. Project description and attributes..... 33

Table 7. User attributes..... 34

Table 8. Differences in access-control semantics..... 36

Table 9. Scenario1 user categories..... 38

Table 10. Scenario2 user categories..... 39

Table 11. Scenario3 user categories..... 40

Table 12. Scenario4 user categories..... 41

Table A1. Design-Science Research Guidelines, based on Hevner et al., (2004, p. 83)..... 60

Table B1. Activities for conducting DSR, based on Peffers et al., (2007, pp. 52, 55-56)..... 61

List of figures

Figure 1. ABAC authorization logic..... 35
Figure 2. RBAC Read access logic.....36
Figure 3. RBAC Write access logic.....36
Figure 4. Scenario1 summarized test results.....44
Figure 5. Scenario2 summarized test results.....45
Figure 6. Scenario3 summarized test results.....46
Figure 7. Scenario4 summarized test results.....47
Figure C1. Validation Test: Design of Test Suite.....62
Figure C2. Validation Test: ABAC-Results.....63
Figure C3. Validation Test: RBAC-Results.....63
Figure D1. Scenario1: Design of Test Suite..... 64
Figure D2. Scenario1: Results.....64
Figure D4. Scenario2: Results.....64
Figure D5. Scenario3: Design of Test Suite..... 65
Figure D6. Scenario3: Results.....65
Figure D7. Scenario4: Design of Test Suite..... 65
Figure D8. Scenario4: Results.....66

1. Introduction

Modern Identity and Access Management (IAM) systems are required to operate within increasingly heterogeneous and complex organizational environments, while also supporting distributed workforces and addressing rising security threats such as remote authentication weaknesses and increased cyber-attacks (Glöckler et al., 2024, p. 421).

From an information systems perspective, IAM systems are best understood as sociotechnical rather than purely technical (Sawyer & Jarrahi, 2015, p. 3). Information systems research highlights that technologies and the people who use them continually shape each other. Meaning that neither can be understood in isolation (Sawyer & Jarrahi, 2015, p. 3-4). In this view, IAM operates within and is influenced by the broader organizational environment, since technologies are embedded in social contexts that both guides how they are used and are reshaped through their use (Sawyer & Jarrahi, 2015, p. 5).

This thesis is conducted at Sogeti Karlstad and at their request is based on the Microsoft Azure suite's IAM platform, called Microsoft Entra (Microsoft, 2025a, *What is Microsoft Entra?*). Sogeti is a technology consulting provider and part of the Capgemini Group (Sogeti, 2026, *About Sogeti*). Because Microsoft Entra is one of the primary identity platforms used within Sogeti's client projects, the thesis investigates Role-Based Access Control (RBAC) and Attribute-Based Access Control (ABAC) specifically in the context of Microsoft Entra.
Apart from the requirement to use Microsoft Entra and the .NET ecosystem, all decisions regarding the thesis focus, methodological approach, system design and implementation details were made independently by the author.

Microsoft, (2025d, *What is azure attribute-based access control (Azure ABAC)?*) describes that Microsoft Entra, which includes Entra ID, provides built-in support for Azure Role-Based Access Control (Azure RBAC). Azure also supports extending RBAC through attribute-based conditions, known as Azure Attribute-Based Access Control (Azure ABAC). Azure ABAC does not replace RBAC, instead it builds on it by allowing role assignments to be filtered through resource and identity attributes (Microsoft, 2025d, *What are the terms of role assignment?*). This filtering is described to allow more fine-grained authorization decisions.

Previous research shows that RBAC has historically dominated enterprise security architectures. During the 1990s, RBAC emerged as the dominant access-control model in organizations (Sandhu, 2001, p. 22). RBAC continues to be widely deployed in modern enterprise environments (Coyne et al., 2011, p. 54; Moratore et al., 2025, p. 126). While RBAC is a widely adopted model, it commonly suffers from operational inefficiencies, including redundant or unused roles and role bloat (Moratore et al., 2025, p. 126). In practice, organizations can maintain tens of thousands of roles, often siloed by department (Moratore et al., 2025, pp. 126, 131). This fragmented landscape of independent managers leads to significant redundancy, with thousands of roles sharing identical user or permissions sets (Moratore et al,. 2025, p. 131).

As organizations outgrow these static role-based structures, Attribute-Based Access Control

(ABAC) is often proposed to provide finer granularity and dynamic policy evaluation that RBAC lacks (Hu et al., 2014, pp. 5-6). However, transitioning to an attribute-based model introduces new challenges related to governance, policy design and technical architecture, which often leads to higher administrative complexity and implementation requirements (Hu et al., 2014, pp. 22-25).

1.1 Purpose

The purpose of this thesis is to investigate and compare RBAC and ABAC as implemented in Microsoft Entra. The comparison focuses on how each model affects:

- Authorization granularity and flexibility
- Administrative complexity and operational management
- Implementation challenges, best practices and standards

A Design Science Research Artefact (see "Introduction to the Methodology" in chapter 3), is developed to enable an empirical and controlled evaluation of both access control models under identical conditions.

1.2 Research questions

- RQ1: What are the practical differences between RBAC and ABAC in Microsoft Entra with respect to authorization granularity and flexibility?
- RQ2: How do RBAC and ABAC differ in terms of implementation complexity and administrative workflow within Microsoft Entra?
- RQ3: How do policy maintenance and change management differ when using RBAC vs ABAC in Entra?

1.3 Scope and Delimitations

This scope is deliberately limited to Microsoft Entra's native capabilities and does not include third-party identity providers or external authorization engines.

The research centers on two complementary components, a literature-based conceptual comparison and an empirical evaluation conducted through a controlled artefact implementation in a Microsoft Entra tenant. The artefact assesses each model through the predefined criteria, granularity, administrative steps and policy complexity while using realistic but artificially constructed scenarios. The thesis is not aimed at evaluating RBAC/ABAC performance metrics beyond administrative workload and policy complexity or developing any production-ready model. It is not either aimed at conducting any security risk assessment or penetration test of Microsoft Entra or the models implemented.

Given these delimitations, the results of this study are intended to provide practical, evidence-based guidance to organizations evaluating access-control strategies in Microsoft Entra. The findings contribute primarily to understanding implementation-level differences between RBAC and ABAC and the trade-offs they introduce.

1.4 Ethical considerations

This study involves technical experimentation and the collection of quantitative measurement data from artefact implementations. No personal data is collected through this study and as such, the primary ethical concerns are not related to data collection. The PoC artefact is conducted in a controlled Microsoft Entra environment that does not contain real user accounts, production data or organizational identities. Only synthetic accounts and test

configurations are used. This ensures that no sensitive operational data is exposed and that the research does not interfere with live security settings or organizational compliance requirements. The implementation and setup in Entra are guided through documentation from Microsoft's official websites. If liberties are taken in the implementation choices, due to not finding any official Microsoft guidance online, it is clearly stated. This ensures that the reader can trust that the implementation is done through best practices and avoid replicating inefficient or insecure code. Since this thesis is not aimed at producing any guidance on how to implement these models, just what trade-offs and implications they might bring, it avoids the risk of a misinterpretation of the findings leading to improper implementation through replication of the PoC. The limitations of the study are clearly and transparently displayed in the thesis to not risk overpromising the findings' implications on real-world scenarios. The four test-scenarios are identical between the models, and the implementation is done through documented best-practices to reduce the risk of a biased implementation that would give misdirected results.

1.4.1 GenAI Disclaimer

Generative AI (GenAI) has been used in the process of conducting the thesis with inspiration about artefact architecture and implementation strategy. It has also been used to generate and discuss ideas about sources and create a time plan for the different components of the thesis based on the limited time-period.

Before implementing the artefact, some exploratory development was done through the use of LLM's to get a feel for the technologies and frameworks. This code was not used when developing the PoC but can have colored the developers work subconsciously.

When implementing the PoC artefact Microsoft Copilot autocomplete has been used to speed up development time, but only as autocomplete suggestions and not to generate solutions or code blocks.

2. Literature overview

This chapter presents the theoretical and technical foundation for the study. It provides an overview of Identity and Access Management (IAM), fundamental access control concepts and the characteristics of Role-Based Access Control (RBAC) and Attribute-Based Access Control (ABAC) as described in the literature.
In addition to conceptual foundations, the chapter reviews prior research related to access control models and their application in modern systems. Particular attention is given to how RBAC and ABAC differ in terms of authorization logic, granularity, administrative complexity and adaptability. The chapter also introduces Microsoft Entra as the contextual platform for this study and highlights the research gap that motivates the empirical evaluation conducted in this thesis.

Below follows a list of used Acronyms with a brief description in table 1:

Table 1. List of Acronyms and Concepts.

Acronym	**Description**
RBAC	Role-Based Access Control: Access control model based on role membership.
ABAC	Attribute-Based Access Control: Access control model based on evaluation of attributes.
IAM	Identity and Access Management: Framework for managing identities and access rights.
API	Application Programming Interface: Interface for communication between software components.
API endpoints	URL where the API can be accessed.
.NET	Microsoft-developed framework for building and running applications.
ASP.NET CORE	Cross-platform framework for building web applications using .NET
OAuth 2.0	Authorization framework enabling secure delegated access to resources.
OpenID Connect	Identity layer built on OAuth 2.0 for user authentication.
JWT	JSON Web Token: Compact token format used to securely transmit information.
Authentication/ID Token	Token containing identity information about the authenticated user.
Access token	Token used by a client to access protected resources.
ROPC	Resource Owner Password Credentials flow: OAuth 2.0 flow where credentials are directly exchanged for tokens.
Authorization Code flow	OAuth 2.0 flow, using an authorization code for secure token exchange, involving interactive login and client redirects.
Microsoft Graph	API used to access Microsoft Entra data and services.

Service Principal	Identity used by applications or services in Entra.
Managed Identity	Managed identity provided by Entra for secure service authentication.
Tenant	Isolated instance of Microsoft Entra representing an organization.

2.1 Identity and Access Management in Cloud Environments

To establish the context for access control models, the chapter begins with an overview of IAM in modern cloud environments.

Within modern, highly connected network environments, Identity and Access Management (IAM) provides mechanisms to verify users, devices and services. Typically incorporating technologies such as multifactor authentication and other identity-based controls to ensure secure access to resources (Microsoft, 2025b, *Identity and Access Management fundamental concepts*).

IAM ensures that the correct entities, whether human users, devices or software components are granted appropriate access to resources at the right time (Microsoft, 2025b, *What is identity and access management*). According to Microsoft (2025b, *What IAM does*), IAM includes identity management, authentication, authorization, access control, provisioning, federation and monitoring.

Authentication and authorization are conceptually distinct processes. Authentication verifies the identity of an entity, while authorization determines what the authenticated entity is allowed to access within a system (Hu et al., 2014, p. 2; Microsoft, 2025b, *Authentication vs. authorization*).

Microsoft Entra illustrates how cloud platforms implement Identity and Access Management (IAM) in practice. It functions as a centralized identity provider that manages users, applications and services within a unified directory (Microsoft, 2025a, *What is Microsoft Entra?*; Microsoft, 2025b, *Identity provider*). A Microsoft Entra tenant represents an organizational instance of Microsoft Entra (Microsoft, 2025a, *Microsoft Entra ID*).

Microsoft Entra's identity model extends beyond human users to include service principals and managed identities, which together form workload identities used by applications and services (Microsoft, 2026c, *What are workload identities?*). This creates a diverse and highly dynamic authorization environment, where access decisions must account for different types of principals and contextual conditions.

In such environments, traditional role-based approaches may become difficult to manage as access requirements grow in complexity (Hu et al., 2014, p. 6). Attribute-Based models offer the potential for more flexible and context-aware policies (Hu et al., 2014, pp. 5-6). This makes Microsoft Entra a relevant and realistic context for comparing RBAC and ABAC and motivates the need to evaluate how these models behave in practice.

2.2 Fundamental Concepts in Access Control

The following section introduces the fundamental concepts of access control.

2.2.1 Core Access Control concepts

Access control regulates which subjects may perform which operations on which objects (Hu et al., 2014, p. 4).
Hu et al. (2014, p. 7) defined several foundational concepts used across access control models. A *Subject* is described to refer to any entity, whether a human or Non-Person Entity (NPE), that initiates a request to perform an action on a protected resource. They describe an *Object* as a resource that is defined as a logical entity that requires protection from unauthorized access (Hu et al., 2014, p. 7). An *operation* is described to encompass actions such as discovering, reading or manipulating objects (Hu et al., 2014, p. 7). A *Policy* is described to represent the rules and logical relationships between the subject, object and possibly contextual environment conditions, to determine the access result of a request (Hu et al., 2014, p. 7). *Authorization* refers to the decision to permit or deny a subject's access to perform operations on an object, based on defined access control policies (Hu et al., 2014, pp. 2, 7).
These elements serve as a common foundation of concepts and represent the basic principles of logical access control that allow ABAC systems to encompass and enforce Discretionary Access Control (DAC), Mandatory Access Control (MAC), and RBAC policies (Hu et al., 2014. pp. vii, 33).

Discretionary Access Control (DAC) allows resource owners to manage and delegate permissions to other users (Sandhu et al., 1996, p. 40). Mandatory Access Control (MAC) enforces centrally defined security classifications. Access decisions are based on the sensitivity levels assigned to users and objects (Sandhu et al., 1996, p. 40).
Sandhu et al., (1996, p. 40) highlights Role-Based Access Control (RBAC) as an advancement over these conventional models. The authors argue that RBAC simplifies the security administration of large-scale systems. Furthermore, its inherent flexibility allows organizations to easily adjust access rights to meet evolving business requirements and complex security needs (Sandhu et al., 1996, pp. 38-40). Hu et al., (2014, p. vii) explains that Attribute-Based Access Control (ABAC) provides a more scalable and adaptable framework than traditional models by evaluating a diverse set of attributes rather than relying solely on static roles or identities. This approach offers superior flexibility and granularity as it allows for precise access decisions based on complex rule sets that can incorporate subject, object and environmental conditions (Hu et al., 2014, pp. vii-viii).

2.2.2 Authorization granularity and flexibility

In the context of logical access control, granularity refers to the level of precision a system uses to evaluate and enforce authorization. According to Hu et al., (2014, pp. 4-5), traditional coarse-grained approaches, such as Access Control Lists (ACLs) or Role-Based Access Control (RBAC) often rely on predefined categories like a user's identity or assigned role to grant access.
In contrast, a fine-grained approach like Attribute-Based Access Control (ABAC) provides a higher degree of differentiation by considering a "higher number of discrete inputs" during the decision-making process (Hu et al., 2014, pp. vii, 6). This allows the system to distinguish precisely between the characteristics of the subject, object and operation being performed, as well as the environment conditions, such as time or location (Hu et al., 2014, p. 7). By

evaluating these diverse variables against a defined policy, the system can produce a more definitive and flexible set of rules for protecting resources (Hu et al., 2014, pp. vii, 6).

In the ABAC model, granularity is achieved by evaluating attributes associated with the subject, object and environment (Hu et al, 2014, p. vii). Hu et al, (2014, pp. 5-7) describe ABAC as an approach in which authorization decisions are derived from name-value attribute pairs, processed through boolean policy rules. Because policies may incorporate numerous attribute combinations, ABAC enables what the authors describe as “precise access control” (Hu et al, 2014, p. 33).

The granularity of access models can therefore be analytically understood along several dimensions. First, it’s fundamentally tied to its precision, which is achieved by incorporating a greater quantity of distinct data points into the authorization process (Hu et al., 2014, p. vii). This level of detail allows the system to partition access with high specificity, moving from broad categories to highly detailed levels, such as limiting permission to members of a particular project or restricting access during specific timeframes (Hu et al., 2014, p. 29). The effectiveness of the model is determined by the variety and depth of attributes accessible for evaluation, which provides a larger set of variables for expressing policy (Hu et al., 2014, pp. vii, 10). The logical complexity of the boolean rules and hierarchical conditions used to assess these variables determines how strictly or flexibly a request is analyzed (Hu et al., 2014, pp. 5, 24). These factors together dictate the model's granular capacity in its access decisions (Hu et al., 2014, pp. vii, 33).

Compared to ABAC, RBAC primarily bases authorization on role membership, which represents a comparatively limited set of attributes (Hu et al, 2014, pp. 5-6). Increasing precision in RBAC often requires introducing additional roles, potentially leading to structural extension. ABAC by contrast, increases decision precision through dynamic attribute evaluation rather than additional static assignments (Hu et al, 2014, pp. 10, 33). Granularity thus serves as a pillar stone for comparing the expressive capabilities of the access models discussed in this thesis.

2.2.3 Administrative complexity and operational management

Administrative complexity in access control can be understood as the cumulative organizational and technical effort required to design, implement and sustain authorization mechanisms over time. It includes not only initial configuration tasks but also ongoing governance activities such as policy maintenance, attribute management and validation processes (Hu et al, 2014, pp. 22-25). In practice, such complexity increases with system scale and reflects how difficult a model is to manage reliably across distributed environments (Hu et al, 2014, p.33).

Hu et al, (2014) highlights that enterprise ABAC deployments require significant upfront planning. Including defining policies, identifying and structuring attributes, establishing authoritative attribute sources and developing architectures and processes for attribute provisioning, retrieval and integrity assurance (Hu et al, 2014, pp. 22-23). As scale increases, enterprise ABAC requires more complex management capabilities to ensure consistent sharing and use of policies and attributes, as well as the controlled distribution and coordinated operation of access control mechanisms across the enterprise (Hu et al, 2014, pp. 11-12, 24-25).
The need for rule conflict resolution, mechanisms to ensure attribute integrity and processes for managing rules and attribute lifecycles highlights the complexity of governance in ABAC

systems (Hu et al, 2014, pp. 23-24). The authors explicitly note that ABAC systems are more complicated and more costly to implement and maintain than simpler models (Hu et al, 2014, p. 33).

While RBAC is regarded as administratively simpler because permissions are grouped into roles that remain relatively stable (Sandhu et al., 1996, p. 38), complexity may emerge differently as organizations scale. As access requirements diversify and become more situational, the system may suffer from "role explosion", where the creation of numerous ad hoc roles to meet specific needs becomes cumbersome to manage (Hu et al., 2014, pp. vii, 6). In such cases, the administrative burden comes from structural growth, with the management of thousands of roles and their interrelationships becoming a "formidable task" that requires complex enterprise governance to prevent fragmented security (Sandhu et al., 1996, pp. 45-46).

2.2.4 Implementation challenges, best practices and standards

The technical challenge of access control has historically involved the need to embed authorization logic directly within individual applications, as early systems required application-specific implementations of access control mechanisms (Sandhu et al., 1996, pp. 38-39). While role-based models were introduced to simplify security administration by decoupling users from permissions, initial implementations still required developers to encode this logic within software components (Sandhu et al., 1996, pp. 38-39).
Modern approaches, such as attribute-based models, enable authorization decisions to be driven by shared, machine-enforceable policies rather than application-specific mechanisms. This requires systems that can convert human defined access rules into computer-enforceable logic and that ensures that all the different authorization components across the system can communicate and work together correctly (Hu et al., 2014, pp. 21-23, 27).
Achieving a successful deployment requires a broad evaluation of how the system impacts technical design, organizational security and the ability of different systems to work together (Hu et al., 2014, p. viii). Hu et al. (2014, p. 33) notes that ABAC systems are more complicated and more costly to implement and maintain than simpler access control approaches.

For RBAC, implementation in the Microsoft identity platform typically involves mapping application functionality to predefined "App Roles". This approach is considered one of the preferred and simplest methods, as it is directly supported by Microsoft's development libraries (SDKs), which provide built-in mechanisms for processing incoming tokens and performing role-based authorization checks (Microsoft, 2024a, *Implement role-based access control*).

In the Microsoft identity platform, authentication is based on OpenID Connect, while authorization is based on OAuth 2.0 (Microsoft, 2025g, *Endpoints*). OAuth 2.0 is an authorization framework that introduces an access layer and separates the client from the resource owner, allowing applications to access protected resources without directly handling user credentials (Hardt, 2012, p. 4). However, OAuth 2.0 does not provide information about the identity of the user. OpenID Connect builds on top of OAuth 2.0 by adding an identity layer, enabling applications to obtain information about the authenticated user and establish login sessions (OAuth.com, n.d.a, *OpenID Connect*).
OpenID Connect introduces the concept of ID tokens, which are security tokens issued by the authorization server that contains information about the authenticated user, allowing the client application to verify the user's identity. These tokens are usually formatted as JSON Web

Tokens (JWTs), which can be validated and interpreted by the application to establish an authenticated session (OAuth.com, n.d.b, *ID Tokens*).

Microsoft (2023, *What is the .NET SDK?*) describes the .NET SDK as a Software Development Kit that consists of a collection of tools and libraries provided by Microsoft. It is designed to support the development of applications and libraries within the .NET platform. In frameworks like ASP.NET Core (see "ASP.NET Core" in chapter 2), this support enables developers to enforce authorization declaratively at the controller level using built-in attributes that validate role membership (Microsoft, 2024a, *ASP.NET Core web API*). As a result, much of the authorization logic is managed by the framework and underlying SDK, reducing the amount of custom code required. Although RBAC can be implemented using framework-supported mechanisms, developers must still incorporate application-level logic to interpret role assignments and enforce access-control decisions based on those roles (Microsoft, 2024a, *Implement role-based access control*).

In contrast, ABAC implemented through policy-based authorization in ASP.NET Core, introduces a different category of effort. While the authorization can still be triggered declaratively using policy names, the underlying logic is significantly more complex (Microsoft, 2026b, *Policy-based authorization in ASP.NET Core*). It requires the explicit creation of requirements and requirement handlers to evaluate dynamic data (Microsoft, 2026b, *Policy-based authorization in ASP.NET Core*). This shifts the complexity from simple role checks into custom code within handlers and potentially external security APIs (see 3.3.1, *System overview*). The implementation burden increases because developers must manage the building blocks of requirements, handlers and the retrieval of external data to make a final authorization decision (Microsoft, 2026b, *Requirements-Authorization handlers*).

These foundational concepts provide the basis for understanding the differences between RBAC and ABAC explored in the following sections.

2.3 Role-Based Access Control (RBAC)

2.3.1 Theoretical Foundations

RBAC was formalized by Sandhu et al., (1996, p. 38), who defined roles as semantic constructs representing organizational functions. Instead of assigning permissions directly to users, permissions are assigned to roles and users are then assigned to those roles. This abstraction simplifies administration because user-role relationships change more frequently than role-permission relationships (Sandhu et al., 1996, p. 38).
RBAC includes four fundamental elements which are *users*, *roles*, *permissions* and *sessions* (Sandhu et al., 1996, p. 41).
Hierarchical RBAC (RBAC1) introduces role inheritance, allowing senior roles to inherit permissions from junior roles (Sandhu et al., 1996, p. 42). This hierarchy supports organizational structures and reduces duplication of permissions.

RBAC supports the key security principles of *Least privilege* by ensuring that roles are assigned "only those permissions required for the tasks" that a user must complete (Sandhu et al., 1996, p. 40). The model also allows users to activate a session that activates only the roles necessary for their current activities, preventing unnecessary access to protected resources (Sandhu et al., 1996, pp. 41-42). RBAC is described to also support the principle of *Separation of duties* by utilizing mutually exclusive roles to carry out sensitive operations

(Sandhu et al., 1996, pp. 39-40). For example, a system can be configured so that different roles are required to participate in sensitive tasks, such as having both an accounting clerk and an account manager be involved in issuing a check (Sandhu et al., 1996, pp. 39-40). Instead of relying on low-level system actions such as *read* or *write*, RBAC allows permissions to be defined as abstract operations representing higher-level business functions. For instance, permissions such as *credit* or *debit* for an account object can be established, reflecting application-specific operations rather than system-level commands (Sandhu et al., 1996, pp. 40-41). By providing these properties, RBAC provides a versatile and robust access control framework, capable of addressing diverse organizational requirements (Sandhu et al., 1996, pp. 39-40).

2.3.2 Administrative Strengths and Limitations

RBAC is widely recognized for its administrative simplicity. By assigning permissions to roles rather than individual users, it centralizes permission management and reduces the complexity of user-permission assignments (Sandhu et al., 1996, p. 38). This structured approach can reduce authorization errors, although ineffective role management may introduce auditing challenges (Moratore et al., 2025, p. 126).

While RBAC provides a structured approach to access control, it faces significant implementation and maintenance challenges as organizations scale. According to Coyne et al., (2011, pp. 54-55), large firms often need to manage a large number of roles, which creates a complex requirement to structure them into efficient hierarchies. This occurs because organizations support a large amount of users and permission controls, which often leads to intricate systems that present security hurdles (Coyne et al., 2011, p. 54). There is also the possibility of creating redundant or inefficient roles. Moratore et al (2025, pp. 126-128) have identified five common inefficiencies, including unused roles, duplicate roles and roles with overlapping permissions. Their framework, when applied to a real, large international organization, showed that consolidating redundant roles could eliminate about 10% of roles (Moratore et al. 2025, p. 127). Traditional RBAC models can also struggle with dynamic environments. The process of role engineering with defining and updating roles is often difficult since departments might use inconsistent naming conventions and the specific permissions required for various job positions are often unclear (Coyne et al., 2011, p. 55). To address the weakness of RBAC in “rapidly changing environments”, Coyne et al., (2011, p. 56) describes that newer standards like the RBAC Policy-Enhanced (RPE) model have been developed to handle “dynamic constraints” that traditional role structures cannot inherently handle. These enhancements are described to allow for “attribute-sensitive” decisions, which reduces the need for constant manual overhaul of roles when contextual factors change (Coyne et al., 2011, p. 56).

2.3.3 RBAC Implementation in Microsoft Entra

Microsoft Entra implements Role-Based Access Control (RBAC) through predefined directory roles that encapsulate administrative permissions (Microsoft, 2025f, *Overview of role-based access control in Microsoft Entra ID*). These *built-in roles* correspond to common organizational responsibilities, such as global administration, security administration and application management (Microsoft, 2026a, *Microsoft Entra built-in roles*). They bundle permissions into functional categories consistent with classical RBAC principles (Microsoft, 2026a, *Microsoft Entra built-in roles*; Sandhu et al., 1996, p. 38). In addition to built-in roles, Entra supports *custom roles*, which allow organizations to define tailored permission sets by selecting from a predefined catalog of actions (Microsoft, 2025f, *Understand Microsoft Entra role-based access control*).

Permissions are granted through role assignments, which associate a role definition with a security principal, such as a user, group or service principal, at a specified scope (Microsoft, 2025f, *Understand Microsoft Entra role-based access control*). The scope defines the extent of control, ranging from tenant-wide access to more limited contexts such as administrative units or individual resources. This enables fine-grained delegation of administrative authorities while preserving centralized control over permission definitions (Microsoft, 2025f, *Scope*).
Together, these elements reflect the RBAC model described by Sandhu et al., 1996, p. 38), where roles are designed to mirror real organizational functions and responsibilities, and access decisions are based on role membership rather than individual user permissions. By structuring access around roles, organizations can simplify administration, enforce least-privilege principles and align technical access control with existing organizational hierarchies (Sandhu et al., 1996, pp. 38-40).

2.3.4 Operation and Architectural Considerations for RBAC

Role-Based Access Control (RBAC) is based on a structured model in which access decisions are derived from defined relationships between users, roles and permissions. In this model, roles are created according to the job functions performed within an organization, and permissions are assigned to these roles (Sandhu et al., 1996, p. 38). Users are then assigned to roles based on their responsibilities, allowing them to inherit the permissions associated with those roles (Sandhu et al., 1996, p. 38).
From an architectural perspective, RBAC is defined through core components and relationships, including user-to-role assignments and role-to permission assignments (Sandhu et al., 1996, p. 38). These relationships form the basis for access control decisions, where a user's ability to perform an action depends on the permissions assigned to their roles. Because permissions are associated with roles rather than individual users, they tend to remain more stable over time, while user-role assignments can be adjusted more frequently, as organizational needs change (Sandhu et al., 1996, p. 38).
However, RBAC introduces additional design and administrative considerations due to its reliance on structured relationships between roles, permissions and users. Organizations define roles according to job functions, establish role hierarchies to reflect lines of authority and assign permissions to roles as part of the access control model (Sandhu et al., 1996, pp. 38-39, 42). While these mechanisms support structured access management, they also require organizations to align roles with organizational responsibilities and to manage role-permission assignments effectively. As systems grow in size and diversity, multi-factor access decisions may be required, which can lead to the proliferation of roles, commonly referred to as "role explosion", increasing administrative overhead and reducing clarity (Hu et al., 2014, p. 6). RBAC also lacks the ability to adapt dynamically to changes in context, meaning that modifications to access requirements typically would require updates to roles or role assignments (Hu et al., 2014, p. 6).
Collectively, these characteristics demonstrate that while RBAC has lower architectural complexity in terms of policy evaluation and system design. It may incur increasing administrative complexity as organizational requirements become more dynamic or context-dependent.

2.4 Attribute-Based Access Control (ABAC)

2.4.1 Theoretical Foundations

Attribute-Based Access Control (ABAC) was formally defined by Hu et al., (2014) as an access control model in which authorization decisions are determined by evaluating attributes associated with the requesting subject, the target object and relevant environmental conditions (Hu et al 2014., p. 7). In contrast to identity or role-centric models, ABAC does not rely on a single attribute type such as identity or role membership. Instead, access decisions are derived from the evaluation of multiple subjects, objects and environmental attributes at the time of the request, enabling context-sensitive and fine-grained control (Hu et al., 2014, pp. 5-6).
Attributes in ABAC are expressed as name-value pairs and may represent simple or complex properties of entities within the system (Hu et al., 2014, p. 7). Policies are defined as logical or boolean expressions that evaluate combinations of these attributes to determine whether a specific operation on a protected object should be permitted (Hu et al., 2014, pp. 7, 10).
Hu et al., (2014, p. 7) describe four foundational components underlying ABAC implementations. These are *Subject attributes* which capture characteristics of the requesting entity, such as organizational affiliation, department, clearance level, current assignment or device properties (Hu et al, 2014, p. 7). *Object attributes* describe properties of the protected resources, such as classification level or ownership (Hu et al, 2014, p. 7). *Environmental attributes* represent contextual information, including time, location, system state or risk level, enabling policies to adapt to changing operational conditions (Hu et al, 2014, p. 7). Together, these components enable ABAC to perform real-time, context-aware authorization decisions.
A key strength of ABAC is its flexibility in supporting access decisions based on multiple subjects, objects and environmental attributes at the time of the request. Unlike role-based approaches that rely on predefined roles and may require the creation of numerous roles to accommodate changing requirements (Hu et al., 2014, pp. 5-6).

2.4.2 Administrative Strengths and Challenges

ABAC offers higher granularity than RBAC because decisions incorporate multiple attribute dimensions. It can also adapt to changes without requiring role redesign (Hu et al., 2014, pp. vii, 5-6). Hu et al. (2014, p. 5) highlights that one of ABAC's strongest advantages is that permissions do not need to be explicitly assigned to specific users or roles in advance, but are derived through attribute evaluation at the time of access. Instead, policies are evaluated dynamically at the moment of access, meaning that new users or external actors can be accommodated without modifying existing rules, given that they possess the required attributes (Hu et al., 2014, pp. 10-11). This makes ABAC highly adaptable in situations where traditional role structures do not reflect operational needs (Hu et al, 2014, pp. 5-6). Another benefit noted by the authors is that because the introduction of new users does not require a policy redesign as long as the attribute values match existing policy requirements, the system continues to operate without additional administrative overhead. This allows the model to scale efficiently, especially in diverse or rapidly changing environments (Hu et al, 2014, pp. 10-11).

Although ABAC offers considerable flexibility, it introduces architectural and administrative complexity, as organizations must design attribute architectures, manage policies and rules and establish governance mechanisms across distributed access control components (Hu et al., 2014, pp. 22-25). This does not contradict the purpose of ABAC, but rather reflects a fundamental trade-off in the model. The flexibility of ABAC arises from its ability to evaluate multiple attributes dynamically at decision time. However, this requires extensive upfront

planning, including identifying protected objects, defining governing policies, establishing attribute structures and determining how access control mechanisms will be distributed across the enterprise (Hu et al, 2014, pp. 22-23). Organizations must implement dependable attribute management practices, such as defining clear attribute schemas, specifying permitted values, designating authoritative sources and implementing controlled procedures for assigning and verifying both subject and object attributes (Hu et al, 2014, pp. 23-24). In addition, policies and rule sets must be carefully administered throughout their lifecycle, including formal development, validation, maintenance and mechanisms for resolving rule conflicts (Hu et al, 2014, p. 24). ABAC systems also depend on coordinated mechanisms for attribute retrieval, policy evaluation, contextual decision handling and enterprise-wide information exchange, all of which can increase technical complexity and implementation costs (Hu et al, 2014, pp. 24-25).

While ABAC improves granularity and contextual responsiveness, its flexibility introduces increased implementation and administrative complexity, making governance and system design more demanding (Hu et al., 2014, p. 33).

2.4.3 Attribute-Based Controls in Microsoft Entra

Although Microsoft Entra is structurally role-based, it incorporates attribute-based elements through conditional role assignments and policy evaluation mechanisms (Microsoft, 2025d, *What are the terms of role assignment?*). Azure Attribute-Based Access Control (Azure ABAC) extends RBAC by allowing conditions to be attached to role assignments. This enables role permissions to be dynamically refined using attributes associated with resources and identities (Microsoft, 2025d, *What are the terms of role assignment?*). These conditions evaluate resource attributes and contextual signals before granting access.

Role assignment conditions enable administrators to restrict permissions based on attributes such as resource properties, storage container names or principal characteristics (Microsoft, 2025d, *What are the terms of role assignment?*). Microsoft Entra also supports *custom security attributes* that are key-value pairs that can be defined and assigned to users or resources (Microsoft, 2024b, *What are custom security attributes in Microsoft Entra ID*). These structured attributes can be used to enable more granular policy definitions and facilitate attribute-based evaluation (Microsoft, 2024b, *What are custom security attributes in Microsoft Entra ID?*).

However, this implementation of ABAC represents a hybrid model, where attribute-based conditions are integrated into a fundamentally role-based architecture. In contrast, the ABAC model in this thesis was implemented as a policy-driven authorization mechanism at the application layer, independent of role assignments.

The distinction is important for the interpretation of the results. While Microsoft Entra's native capabilities combine RBAC and attribute-based conditions, the purpose of this study is to compare the underlying models in isolation. Therefore, RBAC and ABAC are implemented as separate mechanisms to enable a controlled evaluation of their respective characteristics, rather than assessing Entra's hybrid authorization features.

2.4.4 Operation and Architectural Considerations for ABAC

Hu et al., (2014, p. 22) emphasizes that successful ABAC deployment requires a structured approach. Organizations must first identify which resources will be governed by ABAC and define policies that govern those objects. They must also define and standardize both subject and object attributes, including establishing authoritative attribute sources and mechanisms for sharing these attributes across the enterprise (Hu et al., 2014, pp. 22-23). Enterprise ABAC implementations must also ensure that access control mechanisms are consistently

distributed and orchestrated across systems for performance and scalability considerations (Hu et al, 2014, pp. 24-25).
Collectively, these requirements demonstrate that while ABAC is highly expressive and flexible, it introduces significantly greater implementation complexity than simpler access control models (Hu et al, 2014, pp. 22-25, 33).

2.5 Hybrid RBAC-ABAC Approaches

Across the literature, a consistent pattern emerges that neither RBAC nor ABAC alone fully satisfies modern organizational requirements.
Hu et al., (2014, pp. 5-6) notes that ABAC is general enough to incorporate RBAC concepts by treating roles as simply another type of attribute. This means that hybrid models, where roles are used for baseline privilege structuring, while attributes refine access at a more granular level, align naturally with the ABAC framework.

Alsaleh & Kaja (2024, pp. 19-20) aimed to address the limitations associated with using RBAC or ABAC in isolation by developing a healthcare prototype that integrates RBAC's structural role management with ABAC's contextual flexibility. Their study indicates that this combined approach supports a more dynamic and adaptable access control system while mitigating limitations inherent in the individual models (Alsaleh & Kaja, 2024, pp. 19-20).
Hamdah & Almoued, (2025, pp. 24-28) apply game theory to model the trade-off between RBAC and ABAC and use Nash equilibrium analysis to identify an optimal policy mix of approximately 63% RBAC and 37% ABAC. In this context, Nash equilibrium represents a stable outcome in which neither the defender nor the attacker can improve their results by unilaterally changing their chosen strategy. Their results show that this hybrid approach reduces breach rates compared to using either model in isolation. While pure RBAC provides stronger structural control, it lacks flexibility. Pure ABAC, although more adaptable, demonstrates higher vulnerability in certain scenarios (Hamdah & Almoued, 2025, pp. 8-10, 26). The authors conclude that hybrid approaches can improve security by combining RBAC's structured controls with ABAC's contextual adaptability (Hamdah & Almoued 2025, pp. 30-31). They also describe several hybrid configurations, including attribute-extended RBAC, dynamic role assignment and treating roles as attributes within ABAC policies (Hamdah & Almoued, 2025, pp. 9-10)

Microsoft Entra provides a practical example of hybrid authorization architecture. Where roles define baseline permissions, while Conditional Access policies and role assignment conditions introduce contextual evaluation (Microsoft, 2025d, *What are the terms of role assignment?*). This layered model demonstrates how commercial cloud platforms combine structural RBAC with attribute-based refinement.
Such hybridization aligns with theoretical hybrid models described in previous studies. However, the coexistence of role hierarchies, conditional policies and custom attributes may increase governance complexity, requiring structured review and lifecycle management processes (Hu et al., 2014, pp. vii, 21-24).
It is important to note that this thesis does not implement or evaluate a hybrid RBAC-ABAC model. Instead, RBAC and ABAC are implemented as separate authorization mechanisms in order to isolate their respective characteristics and enable a controlled comparison. The

discussion of hybrid models is therefore included to provide context for how these approaches are applied in real-world systems, rather than to propose a solution within this study, which is left as an area for future work.

2.6 Microsoft Entra as an IAM platform

Microsoft (2025a, *What is Microsoft Entra?*) describes Microsoft Entra as a family of identity and network access solutions designed to support Zero Trust strategies. Microsoft Entra ID provides identity, authentication, authorization and policy management services (Microsoft, 2025a, *Microsoft Entra ID*). Identity and Access Management (IAM) functionality within Entra includes identity management, federation, provisioning, authentication, authorization and monitoring (Microsoft, 2025b, *What IAM does;* Microsoft, 2025a, *Microsoft Entra ID*). Application management allows administrators to configure access policies, user consent and automated provisioning workflows (Microsoft, 2025c, *What is application management in Microsoft Entra ID?*).
Microsoft Entra supports access control with RBAC through natively built-in roles and role assignments (Microsoft, 2025f, *Overview of role-based access control in Microsoft Entra ID;* Microsoft 2026a, *Microsoft Entra built-in roles*). It also introduces attribute-based conditions and custom security attributes that extend the role-based model (Microsoft, 2025d, *What are the terms of role assignment?*; Microsoft, 2024b, *What are custom security attributes in Microsoft Entra ID*).

While Microsoft's documentation explains Azure RBAC and attribute-based conditions in Entra (Microsoft, 2025d, *What is azure attribute-based access control (Azure ABAC)?*), this thesis's literature review did not identify any peer-reviewed academic studies that evaluate RBAC and ABAC specifically in the context of Microsoft Entra. This suggests a gap in evidence-based guidance for organizations choosing between the models in Entra. This thesis is meant to address that gap.

2.7 Implementation specific frameworks and technologies

In this section the different frameworks and technologies used when developing the artefact according to the DSR approach (see "Introduction to the Methodology" in chapter 3) will be clearly described.

2.7.1 Containerization with Docker

Containerization technologies such as Docker are commonly used to improve reproducibility in experimental software environments. Docker provides lightweight virtualization by packaging applications and their dependencies into self-contained containers. These containers are built from reusable images and support portable and interoperable application deployment across cloud environments (Docker, n.d, *The Docker platform*; Pahl et al., 2019, pp. 677-678).
Containerization supports repeatable testing by allowing applications to be deployed from identical images, eliminating variations in system configuration (Pahl et al., 2019, p. 678). This is particularly valuable in experimental settings, where consistent initial conditions are required for reliable comparison.

Due to time constraints, containerization was not implemented in this thesis. The artefact was instead executed in a controlled local development environment. However, Docker-based

deployment could be considered as a future improvement to further enhance reproducibility and portability of the experimental setup.

2.7.2 ASP.NET Core

Microsoft (n.d.a, *What is ASP.NET Core?*) describes *ASP.NET Core* as a modern, performance-oriented web development framework within the .NET ecosystem. It is the open-source successor to earlier *ASP.NET*, and designed to operate consistently across multiple operating systems and containerised environments (Microsoft, n.d.a, *What is ASP.NET Core?*). These characteristics make it a practical choice for the artefacts development, where predictable execution is essential.

The .NET platform is described as a freely available, open-source runtime and development environment maintained by Microsoft, which offers a secure and efficient foundation for application development (Microsoft, n.d.b, *What is .NET?*). Applications built on .NET commonly use the language C#, a statically typed language that incorporates automatic memory handling and built-in support for concurrent execution, which helps reduce the likelihood of runtime errors in complex application logic (Microsoft, n.d.b, *.NET*).

In addition to the runtime environment, .NET also provides an extensive collection of standardised libraries and application programming interfaces, enabling developers to implement functionality without reliance on custom, system-specific code (Microsoft, n.d.b, *Libraries*). This contributes to consistent dependency handling across different implementations and supports controlled experimentation in comparative studies.

Both .NET and ASP.NET Core are designed to support cross-platform execution and container-based deployment, allowing applications to be developed and executed under identical conditions regardless of the underlying host system (Microsoft, n.d.a, *Windows, Linux, macOS, and Docker*; Microsoft, n.d.b, *Cross Platform*). This is particularly relevant for this study, as it ensures that implementations of RBAC and ABAC within Microsoft Entra can be evaluated under equivalent technical circumstances. Differences in access control granularity, administrative overhead and implementation effort can therefore be attributed to the access-control models themselves, and not to inconsistencies in the application platform.

2.8 Synthesis and Research Gap

The reviewed literature demonstrates a broad consensus regarding the fundamental characteristics of RBAC and ABAC. RBAC is consistently described as simplifying access control administration by structuring permissions around roles that correspond to organization functions, thereby reducing the complexity of direct user-permission relationships (Sandhu et al., 1996, p. 38; Moratore el al., 2025, p. 126).

In contrast, ABAC enables fine-grained, context-aware authorization decisions by evaluating multiple subjects, objects and environmental attributes at the time of the access request (Hu et al., 2014, pp. 5-6). This capability allows ABAC to support highly flexible and dynamic access control requirements, particularly in environments where access decisions depend on contextual or situational factors (Hu et al., 2014, pp. 5-6). However, the literature equally emphasizes that this flexibility introduces increased architectural and administrative complexity, particularly in large-scale enterprise deployments where policies, attributes and access control mechanisms must be coordinated and governed across distributed systems (Hu et al., 2014, pp. vii, 22-25, 33).

Recent research further suggests that hybrid approaches combining RBAC and ABAC can better address the limitations of each model in complex and dynamic environments. For example, Alsaleh & Kaja, (2024, pp. 19-20) develop a healthcare prototype that integrates

RBAC's structured role management with ABAC's contextual flexibility, highlighting a more adaptable and context-aware access control approach, while mitigating challenges associated with each model in isolation. Similarly, Hamdah & Almoued, (2025, pp. 30-31, 33) show that hybrid configurations may achieve a more optimal balance between flexibility and administrative overhead than either model in isolation.
While there is broad agreement on the strengths and limitations of both models, there is less consensus regarding the continued sufficiency of RBAC in modern systems. Sandhu et al., (1996, pp. 38-40) present RBAC as a robust and extensible model capable of supporting organizational access control through the introduction of role hierarchies and constraints. However, later research indicates that RBAC alone may struggle to scale effectively in environments characterized by dynamic access requirements, where the need for fine-grained and context-sensitive decisions can lead to role proliferation and an increased administrative burden (Hu et al., 2014, pp. vii, 6).

Despite the extensive body of theoretical and conceptual research on RBAC and ABAC, the literature review conducted in this thesis identifies a clear gap in empirical studies examining how these models are implemented and managed in modern cloud identity platforms, specifically Microsoft Entra.
In particular, there is a lack of empirical evidence addressing how these access control models differ in terms of:

- Authorization granularity and expressiveness.
- Implementation complexity at the application level.
- Administrative and governance requirements in practice.
- Flexibility and adaptability under controlled conditions.

To address this gap, this thesis conducts an empirical evaluation in a controlled experimental environment, where RBAC and ABAC are implemented as separate authorization mechanisms within the same Microsoft Entra-based system (see Chapter 3-4).

2.9 Critical Review of the Literature

While the reviewed literature provides a strong theoretical foundation for understanding RBAC and ABAC. It's important to critically evaluate its scope, limitations and applicability to modern cloud environments.

A large portion of the foundational descriptions of RBAC and ABAC in this thesis is based on Sandhu et al., (1996) and Hu et al., (2014). These works are highly influential and widely cited, but they primarily define conceptual models and theoretical frameworks, rather than empirically evaluating their behavior in practical implementations. As a result, they provide limited insight into how these models perform in real-world systems, particularly within modern cloud-based platforms such as Microsoft Entra.

Additionally, some of the RBAC literature originates from an earlier technological context, where system architectures and organizational requirements differed significantly from today's distributed and dynamic environments. While the core principles remain relevant, assumptions regarding system stability and role structures may not fully reflect current cloud-based access-control challenges.

The ABAC literature, while more recent, emphasizes the flexibility and scalability of attribute-based models, while also acknowledging their increased complexity in terms of

implementation and governance (Hu et al., 2014). However, many of these claims are based on theoretical reasoning or architectural analysis, rather than empirical validation in operational environments.

Furthermore, industry sources, such as Microsoft documentation, provide valuable insight into practical implementation and platform-specific behavior. However, these sources are not peer-reviewed and may reflect vendor-specific perspectives or design choices that are optimized for particular platforms rather than broadly generalizable across different environments.

Some of the literature related to hybrid RBAC-ABAC approaches is based on applied studies and student theses (e.g. Alsaleh & Kaja, 2024; Hamdah & Almoued, 2025). While these studies contribute useful insights and practical perspectives, they are not peer-reviewed and therefore lack the same level of validation and generalizability as established academic research. Their findings should therefore be interpreted with appropriate caution and in combination with more established sources.

Overall, the literature provides a strong conceptual understanding of access control models, but lacks empirical evaluation in specific modern platforms such as Microsoft Entra. This limitation motivates the experimental approach taken in this thesis, where RBAC and ABAC are implemented and evaluated under controlled conditions to provide practical insight into their real-world behavior.

3. Method

This Chapter describes the methodological approaches used to investigate and compare Role-Based Access Control (RBAC) and Attribute-Based Access Control (ABAC) in Microsoft Entra. The study is conducted as an empirical evaluation in a controlled experimental setting, using a developed Proof-of-Concept (PoC) artefact. This approach allows for a structured and reproducible comparison of access control behavior under identical conditions.
The methodological choices are guided by the research questions, which focus on identifying practical differences between RBAC and ABAC in terms of authorization granularity, administrative complexity, implementation characteristics and policy adaptability.

The chapter begins by outlining the overall research design and rationale. It then gives a full system overview of the PoC artefact implementation and a description of its architecture. It then describes the data-collection and measurement strategy, outlining how empirical observations were gathered. The analysis methods are then presented, including the evaluation criteria used to compare the two models. Finally, the chapter discusses considerations related to reliability, validity and limitations.

3.1 Introduction to the Methodology

This study adopts Design Science Research (DSR) as its overarching research strategy. DSR focuses on the creation and evaluation of IT artefacts to address organizational problems and generate practically useful knowledge (Hevner et al., 2004, p. 77). According to Hevner et al., (2004, p. 82), DSR is based on the principle that knowledge about a problem domain is advanced through the construction and evaluation of an artefact that addresses a specific problem in that domain.
The aim of this thesis is to compare RBAC and ABAC within a Microsoft Entra-based environment, with respect to administrative complexity, granularity and implementation complexity. Hevner et al., (2004, p. 83) describe seven guidelines for DSR, shown in Appendix A. These guidelines were followed in this thesis when developing the methodology approach and artefact design (see Table 3).

Peffers et al., (2007, pp. 46-48) contributes with a design science research methodology (DSRM) that is used for conducting, presenting and evaluating DSR in the context of information systems. Peffers et al., (2007, pp. 52, 55-56) proposes to follow specific activities in developing a design science artefact and they are provided in Appendix B. The activities were followed in this thesis, and the specific ways are described in Table 2.

In accordance with Hevner et al., (2004, p. 87) guideline 4, this study is also designed to provide clear research contributions in terms of both the developed artefact and the evaluation methodology. The artefact enables controlled comparison of access control models, while the structured evaluation approach contributes methodological knowledge. In line with Hevner et al., (2004, pp. 87-88) guideline 5, rigor in this study is ensured through the systematic use of established theoretical foundations and methodological principles. The design and implementation of the artefact build on existing knowledge in access control models and Microsoft documentation and best practices.

The evaluation relies on structured experimental methods and repeatable measurement procedures. Taken together, the methodology of this study aligns with the core principles of Design Science Research as defined by Hevner et al., (2004). The study introduces a purposeful artefact, addresses a relevant problem, applies structured evaluation methods, ensures methodological rigor and contributes both practically and methodologically. The iterative design process and structured communication further support the completeness of the DSR approach of the study.

Table 2. How Peffers et al., (2007, pp. 52, 55-56) Activities for conducting DSR was followed in this thesis.

Activities	**How they were followed**
Activity 1: Problem identification and motivation	The definition and argumentation for the research problem was done in chapter 1, but this was iterated upon and updated after learning more about the problem area from the literature study in chapter 2. This made sure that the researcher had a better understanding of the problem domain and its potential solutions before starting the implementation.
Activity 2: Define the objectives for a solution.	The objectives were defined based on the research questions, focusing on comparing RBAC and ABAC in terms of granularity, administrative complexity and implementation characteristics within Microsoft Entra
Activity 3: Design and Develop	The design process is laid out in this thesis methodology chapter. The implementation was carried out after a thorough design process and is detailed in chapter 4.
Activity 4: Demonstration	The demonstration is done through presentation of measurement data in Chapter 5, that help explain differences in granularity, administrative complexity and implementation complexity for the different models.
Activity 5: Evaluation	The artefact was evaluated through controlled scenario-based testing, where authorization outcomes and system behavior were systematically observed and compared across RBAC and ABAC implementations.
Activity 6: Communication	The communication of the DSR findings are done through this bachelor thesis. Professional audiences may find the trade-off analysis in chapter 6 valuable for implementation decisions.

Table 3. How Hevner et al., (2004, p. 83) guidelines for DSR were followed in this thesis.

Guideline	**How it was followed**
Guideline 1: Design as an Artifact	This thesis DSR methodology resulted in a concrete artifact in the form of a functioning PoC for comparative analysis of the access models (RBAC & ABAC) with Microsoft Entra.
Guideline 2: Problem Relevance	The problem area is described in chapter 1 (Introduction) and includes organizations' difficulty in understanding and making decisions around access model implementation, for applications using Microsoft Entra.
Guideline 3: Design Evaluation	The artifact is evaluated through collected measurement data, using both empirical scenario-based evaluation and structural analysis of the authorization implementations.
Guideline 4: Research Contributions	This study contributes with an artifact (PoC system) and a structured evaluation methodology. The foundational knowledge supporting the PoC development and usefulness is described in chapters 1 & 2. And the methodologies used are clearly stated in chapter 3.
Guideline 5: Research Rigor	Rigor was ensured through the explicitly defined PoC architecture, repeated test suite runs and through standardized automatic measurement data collection.
Guideline 6: Design as a Search Process	The artifact was designed in iterations and simplifications were used when needed.
Guideline 7: Communication of Research	The findings of the artifact are presented both to technical readers through the implementation of the PoC in chapter 4. And managerial stakeholders are provided with a trade-off analysis in chapter 6.

To enable a systematic and controlled comparison, a proof-of-concept (PoC) artefact was developed consisting of a backend application implementing parallel RBAC and ABAC authorization mechanisms. These mechanisms are exposed through identical API endpoints (see 3.3.1, *System overview)* and evaluated under identical conditions. The artefact includes instrumentation for collecting detailed measurement data for each authorization request (see 3.3.4, *MeasurementService)*.

The study is conducted as an empirical evaluation in a controlled experimental setting, where predefined scenarios are executed to observe how each model behaves under different conditions. This approach aligns with Hevner et al., (2004, pp. 82-84) first DSR guideline, which states that research must produce a purposeful IT artefact.

Instead of relying on statistical hypothesis testing, the evaluation focuses on structured observation of authorization decisions, visible resource access and policy behavior.
The problem addressed in this study, the selection and implementation of access control models in cloud IAM systems, is highly relevant in modern organizational contexts. Especially since increasing system complexity challenges traditional authorization models. This supports Hevner et al., (2004, pp. 84-85) second DSR guideline, which states that the

goal of DSR research is to acquire knowledge that enables the implementation of a solution to an important business problem.

3.2 Research design

This study follows a comparative research design, where two access control models (RBAC and ABAC) are analyzed under controlled conditions. According to Lijphart (1971, pp. 683-684), the comparative method is used to analyze relationships and patterns across a limited number of cases, particularly in situations where statistical analysis is not feasible. The study adopts a case-oriented comparative approach, where each model is analyzed as a complete system rather than as isolated variables. As emphasized by Ragin, (2014, p. 35), case-oriented comparison focuses on understanding how different elements within a case interact to produce outcomes. This is particularly relevant for access control models, where authorization decisions are influenced by the interaction between configuration structures, policy logic and contextual conditions.

To support systematic comparison, predefined test scenarios are executed across both RBAC and ABAC implementations. These scenarios act as controlled experimental conditions, allowing for consistent observation of authorization behavior. The goal is not to establish statistical relationships, but to identify and explain observable differences in how the models behave under equivalent conditions. The design therefore combines empirical observations with a controlled experimental structure.

The development of the PoC artefact follows an iterative design process consistent with Hevner et al., (2004, pp. 88-90) DSR guideline 6, where the artefact is gradually refined to achieve functional comparability between the RBAC and ABAC implementations. Simplifications were introduced where appropriate to isolate the authorization behavior under investigation. In particular, authentication was implemented using the OAuth 2.0 Resource Owner Password Credentials (ROPC) flow instead of the Authorization Code Flow. The ROPC flow allows a client to obtain an access token directly by using the resource owner's credentials, without requiring an interactive authorization step (Hardt, 2012, p. 9). This behavior makes it suitable for automated and repeatable test execution scenarios where interactive login is not required. In contrast, the Authorization Code Flow requires user interaction through a user-agent (typically a web browser), where the resource owner is redirected to an authorization server to authenticate and grant access (Hardt, 2012, p. 8). This interactive and redirect-based process makes it less suitable for controlled and automated testing scenarios.
An access token is a credential used by a client to access protected resources. It is issued by an authorization server and represents the authorization granted to the client, including specific scopes and duration of access (Hardt, 2012, p. 10). This enables the resource server to evaluate whether a request should be permitted or not.

The use of ROPC improves the experimental control and reproducibility of the test scenarios. However, it also introduces limitations in terms of realism, as it does not fully reflect recommended production authentication practices. This limitation is permitted in the study since its focus lies in the authorization mechanisms, rather than the authentication flow.
Additionally, containerization using Docker was considered to further improve reproducibility of the PoC artefact environment. It was however not implemented due to time constraints. The system was instead executed in a controlled local environment.

These design choices align with the principle that researchers may simplify or isolate real-world systems to focus on the core phenomena under investigation (Hevner et al., 2004, pp. 88-89).

3.3 Implementation (PoC architecture)

3.3.1 System overview

According to Mozilla Developer Network (2026, *Overview of HTTP*), HTTP serves as the backbone for web-based data exchange. Functioning as a client/server protocol, where the client initiates a request and the server provides the response. HTTP methods, such as *GET* and *POST*, specify which type of action the client intends to perform. For instance, the *GET* method is commonly used to retrieve data from the server, while the *POST* method is typically employed to submit data to the server (Mozilla Developer Network, 2026, *HTTP Messages*).
The artefact developed for this thesis was implemented as an ASP.NET Core backend application that exposes a set of API endpoints for interacting with artificial project data. In this context, a backend application refers to the server-side part of a system responsible for processing requests, executing application logic, interacting with databases and returning responses to the client. An API endpoint represents a callable interface (e.g., a URL combined with an HTTP method such as *GET* or *POST*) through which clients can request data or perform operations. The application is structured using controllers, which group related endpoints and actions that represent individual operations within those controllers. For example, a controller may provide endpoints for retrieving or modifying project data, while each action defines the logic executed for that specific request to those endpoints.

The system contains two parallel authorization pipelines, one for RBAC and one for ABAC. The architecture was intentionally designed so that all other components, such as routing, persistence of database data, authentication and measurements, remain identical across both models. This ensures that authorization logic is the only variable under evaluation, allowing for a controlled comparison. The application uses a SQLite database to store project data and measurement results collected during test execution. A dedicated measurement component records information about each authorization request, including decision outcomes, evaluation characteristics and contextual metrics (see 3.3.4, *MeasurementService*).

The system was executed in a controlled local environment on a single machine. Test execution was automated through Bash scripts, which are command-line scripts used to execute predefined sequences of commands and automate repetitive tasks. The scripts initiated API calls against the application, allowing scenarios to be executed consistently across repeated runs. Reproducibility was maintained through deterministic test execution, controlled configuration and database reseeding between scenarios. To view the source code, visit the GitHub repo[1]. The project's *Readme.md* file explains the project structure and where the different parts are located.

3.3.2 RBAC Implementation

In this thesis, Role-Based Access Control (RBAC) is implemented within an ASP.NET Core application using Microsoft Entra as the identity provider. RBAC is used to control access to

[1] https://github.com/Melkchri100/EntraPoC

project-related resources by assigning users to predefined application roles. These roles are configured as *App Roles* within the Microsoft Entra application registration. This method was chosen since it aligns with Microsoft's documentation where it is presented as the most straightforward approach for developers (Microsoft, 2024a, *Implement role-based access control*). Each role represents a specific set of permissions, such as viewing or modifying project data. When a user authenticates, their assigned roles are included as *claims* in the issued access token, which is then used by the application to make authorization decisions (Microsoft, 2024a, *Define app roles*).

Within the ASP.NET Core application, authorization is enforced using built-in framework mechanisms. Access control is defined at the endpoint level, meaning that each API endpoint specifies which roles are allowed to execute a given operation. This is implemented using built-in authorization attributes, which check the user's role claims before allowing access to the associated action on that endpoint.
When a request is received, the application evaluates the roles contained in the user's *access token* against the required roles for the requested endpoint. Access is granted only if the user holds the appropriate role or roles.
This approach establishes a clear separation of responsibilities. Microsoft Entra manages identity and role assignments, while the application handles enforcement of authorization decisions. As a result, changes to user permissions can be performed administratively within Entra without requiring modifications to the application code.
While this role-based implementation aligns closely with Microsoft Entra's native capabilities, it limits authorization expressiveness. Achieving higher granularity requires the introduction of additional roles to represent combinations of access conditions. This can increase administrative overhead and reduce flexibility as the authorization domain grows.

3.3.3 ABAC Implementation

Attribute-Based Access Control (ABAC) is implemented using the policy-based authorization framework provided by ASP.NET Core, as described in Microsoft's documentation (Microsoft, 2026b, *Policy-based authorization in ASP.NET Core*). This framework enables authorization logic to be defined separately from application code through structured policies.
In this context, a policy represents a set of conditions that must be satisfied for access to be granted. These conditions are expressed as authorization requirements, which define specific access rules. Each requirement is evaluated by an authorization handler, which contains the logic used to determine whether the requirement is fulfilled (Microsoft, 2026b, *Authorization handlers*).
During authorization, the application evaluates whether a request satisfies a defined policy. This process is coordinated by the *IAuthorizationService*, which checks whether the authenticated user meets the required conditions for accessing a resource (Microsoft, 2026b, *IAuthorizationService*).
Custom authorization requirements were implemented to represent specific access-control conditions, such as ownership, sensitivity or attribute-based restrictions. Each requirement is paired with a corresponding authorization handler that performs the necessary evaluation. When a request is processed, the handler receives contextual information through an *AuthorizationHandlerContext*, which includes the authenticated user's attributes, the requested resource, and set of requirements that must be evaluated (Microsoft, 2026b, *IAuthorizationService*). Based on this information, the handler determines whether the required conditions are satisfied. If a condition is met, the handler marks the requirement as successful, allowing the authorization process to proceed.

This approach allows authorization decisions to be evaluated dynamically at runtime, based on both user attributes and resource-specific data. Unlike RBAC, where access decisions are based on static role assignments, ABAC evaluates multiple conditions per request, enabling fine-grained and context-dependent access control.
By structuring authorization around policies, requirements and handlers, the ASP.NET Core authorization framework provides a modular and reusable approach to defining and evaluating access decisions, allowing flexible and extensible authorization logic (Microsoft, 2026b, *Policy-based authorization in ASP.NET Core*).

3.3.4 MeasurementService

To support empirical evaluation of the authorization models, a dedicated component called the "MeasurementService" was implemented within the application. The purpose of this component is to record observable characteristics of each authorization request during test execution.

The *MeasurementService* captures structured data for each request, reflecting how authorization decisions are made rather than only timing or performance metrics. The collected data includes indicators such as the number of authorization decisions evaluated, whether access was granted or denied and how many resources were visible to the user.
The full list of measurement variables is shown in Table 4.
These measurements are derived directly from the execution of predefined test scenarios and are stored in a SQLite database for later analysis. Each test-scenario finished by exporting the measurements to a CSV file before resetting the database. When the database resets it also reseeds with the predefined project resources. This is done between runs of the different scenarios to preserve the starting state between executions and improve the test results validity. The *MeasurementService* operates independently of the authorization logic itself, ensuring that measurement collection does not influence authorization behavior.
Rather than focusing on statistical performance evaluation, the service supports structured observation of system behavior in a controlled environment. The collected data enables comparison of RBAC and ABAC in terms of granularity, decision behavior and structural differences in how authorization is evaluated.
The *MeasurementService* therefore provides the empirical foundation of the study, enabling consistent and reproducible analysis of authorization behavior across both models.

Table 4. Measurement Variables collected from the artefact.

Measurement Variables	**What is evaluated**	**Values**	**Notes**
model	What model enforced authorization	“RBAC”, “ABAC”	Set explicitly by the controller. Never inferred by middleware. Always populated for successful requests.
requestSignature	What request was evaluated	“GET:/rbac/projects”, “GET:/abac/projects/{id}”	Stable, route-level identifier Independent of test scripts Enables per-endpoint comparison.
decisionCount	How many authorization decisions were evaluated	RBAC = Number of authorization constraints evaluated (typically 1) ABAC = Number of resources evaluated (one decision per resource + 1 for endpoint access)	Indicator of authorization granularity and evaluation scope.
impactCount	How much the authorization policy affected visibility	RBAC = Number of restrictive authorization constraints applied ABAC = Number of resources denied by attribute evaluation	Defined differently across models, used for qualitative interpretation only. Indicates how authorization decisions affect resources within each model.
visibleResourceCount	How many resources the user ultimately saw	RBAC = Number of resources returned after role-based filtering ABAC = Number of resources returned after attribute-based evaluation	Represents the observable result of authorization decisions from the user’s perspective.
accessGranted	Whether the request itself was allowed	false → hard deny (403) true → request allowed (possibly filtered)	Indicates whether the request is permitted at the endpoint level.
primaryDenialReason	Most significant restrictive constraint applied	RBAC = Restrictive role or policy that applied ABAC = Attribute condition denying the most resources	Qualitative indicator of the most significant constraint affecting access.
denialReasonSummaryJson	Breakdown of all denial reasons observed	RBAC = Which authorization constraints fired ABAC = Distribution of per-resource denial reasons	Provides detailed breakdown of all contributing denial factors.
executionTimeMs	End-to-end authorization-inclusive request latency	“33ms”	Included for completeness, not used in comparative analysis.
graphApiCalls	Number of live directory (Microsoft Graph) calls	RBAC = 0 → static/token based ABAC = > 0 → Dynamic	Indicator of runtime dependency on external attribute retrieval and evaluation.

The collected metrics serve different analytical purposes. The primary indicators used for cross-model comparison are *DecisionsCount, VisibleResourceCount* and *AccessGranted*, as these provide consistent and comparable measures of authorization behavior across both RBAC and ABAC.
Other metrics, such as *ImpactCount, denialReasonSummaryJson, graphApiCalls* and *executionTimeMs,* are used for contextual and explanatory purposes. These metrics provide additional insight into how authorization decisions are applied within each model, but are not used as primary indicators for comparison. This ensures that observed differences can be attributed to the authorization models themselves, rather than inconsistencies in measurement or system state.

3.3.5 Test Scenarios

The study uses four predefined test-scenarios, which are executed identically for both RBAC and ABAC implementations to ensure consistent and controlled comparison.
The scenarios are designed to reflect authorization behavior observable at application runtime, rather than administrative operations within Microsoft Entra. This ensures that the evaluation focuses on how each model enforces access control under equivalent conditions.
Each scenario isolates a specific aspect of authorization behavior, such as resource visibility, sensitivity handling, ownership conditions and mutation capabilities. This allows differences between RBAC and ABAC to be attributed to model characteristics rather than environmental variation. The scenarios are explained in detail in Chapter 4 and listed briefly in Table 5.

Table 5. Test-scenarios used for automated tests.

Scenario	**Description**
S1: Read-heavy project listing with filtered visibility	Focuses on visibility semantics by evaluating which resources are returned to the user. RBAC determines access through role-based filtering. ABAC evaluates access per resource using attribute-based conditions, resulting in different visibility outcomes.
S2: Sensitive resource access control	Evaluate access to sensitive projects. RBAC requires additional roles to grant access, while ABAC determines access dynamically based on resource sensitivity and user attributes.
S3: Ownership-based access overrides	Evaluates how ownership affects authorization decisions. RBAC does not natively support ownership-based access overrides. Whereas ABAC enables dynamic access decisions based on ownership attributes and contextual policy conditions.
S4: Write operations (create, update, delete)	Evaluates authorization for modification operations. RBAC requires explicit role combinations to permit write access. ABAC evaluates multiple conditions at runtime, enabling more flexible access control.

3.4 Data Collection Methods

3.4.1 Automated testing

To support the empirical evaluation of RBAC and ABAC, automated tests were used to execute predefined authorization scenarios in a consistent and controlled manner. These tests were designed to simulate realistic access requests and to ensure that both authorization models were evaluated under identical conditions.
Although the automated tests were not implemented as isolated unit tests, they were designed with attention to reliability and repeatability principles, as highlighted in Microsoft's testing guidance (Microsoft, 2025e, *Unit testing best practices for .NET*). In particular, efforts were made to ensure deterministic outcomes, meaning that repeated execution of the same scenario yields consistent results when the system configuration remains unchanged.
To achieve this, each scenario's test-script was executed in isolation and relied on controlled inputs. The application state was standardized between test runs by using a fixed dataset and resetting the underlying database state prior to execution. This ensured that all scenarios were evaluated from the same starting conditions. External variability was minimized by avoiding unnecessary dependencies on external systems. Authentication and authorization inputs were controlled, and all requests were executed programmatically to ensure consistency across runs.

The test suite was structured around clearly defined scenarios, where each test case focused on a specific aspect of authorization behavior. This approach supports repeatability and allows observed differences between RBAC and ABAC to be attributed to the authorization models rather than environmental variation.
The use of automated testing therefore provides a stable and reproducible foundation for data collection, enabling consistent observation of authorization decisions and system behavior across multiple test executions.

3.4.2 Quantitative Measurement Collection

Quantitative research uses systematically collected measurement data and implements statistical methods during analysis (Patel & Davidsson, 2019, p. 51). Its primary aim is to address questions related to variable magnitude and frequency, as well as the relationships and causal connections between these variables. Furthermore, the quantitative approach centers on phenomena that can be observed and quantified and seeks to maintain a high degree of objectivity throughout the research process (Patel & Davidsson, 2019, p. 53).

The data collection in this study consists of structured observations derived from the execution of predefined authorization scenarios within the Proof-of-Concept (PoC) artefact environment. Rather than following a strictly quantitative research approach, this study applies an empirical measurement strategy, focused on observing and comparing system behavior under controlled conditions. Measurement data is generated automatically during test execution through the implemented *MeasurementService*, which records key characteristics of each authorization request. These measurements do not aim to provide statistical inference, but rather to capture observable differences in how RBAC and ABAC process access decisions.
The collected data primarily reflects three dimensions aligned with the research questions.
Granularity is evaluated through observable authorization behavior. RBAC typically produces a single access decision for an entire request based on role membership, while ABAC

evaluates access separately for each resource, using attribute-based conditions. As a result, ABAC can produce more varied and fine-grained access outcomes within the same scenario.
Administrative complexity is not measured through runtime data. Instead, it is evaluated by examining how access control must be configured and maintained in each model, including how roles, attributes and policies are created, modified and managed.
Implementation complexity is evaluated by comparing how authorization logic is implemented in practice. This includes how access rules are expressed in code, how decisions are evaluated at runtime and whether access control must be configured and maintained in each model, including how roles, attributes and policies are created, modified and changed.
All measurements are derived from scenario-based execution in a controlled environment, ensuring that observed differences between RBAC and ABAC can be attributed to the authorization models themselves, rather than variations in system configuration.
The measurement framework therefore supports a structured comparison of authorization behavior, providing a practical and reproducible basis for evaluating the differences between RBAC and ABAC.

3.5 Data Analysis Methods

In design science research, evaluation plays a central role by demonstrating the utility, quality and effectiveness of the constructed artefact (Hevner et al., 2004, pp. 85-87). In this study, evaluation is not aimed at statistical generalization, but at systematically assessing how well the artefact supports the comparison of RBAC and ABAC under controlled conditions.
The data analysis in this thesis follows a structured empirical approach combining scenario-based evaluation, comparative analysis and design science evaluation principles.

In addition to its technical focus, the analysis is informed by a sociotechnical perspective on Identity and Access Management, as introduced in Chapter 1. IAM systems do not operate in isolation as purely technical constructs, but are embedded within organizational and social contexts, where technologies and human actors are mutually constituted and shaped by norms, rules of use and organizational practices (Sawyer & Jarrahi, 2015, pp. 3-5). In this sense, IAM elements such as users, roles, policies and attributes can be understood as reflecting underlying organizational structures and governance processes.
Consequently, the observed differences between RBAC and ABAC are not interpreted solely as differences in authorization logic, but also as differences in how each model represents and manages organizational relationships, responsibilities and contextual information. Role assignments in RBAC reflect formalized organizational structures (Sandhu et al., 1996, pp. 38, 41), while attribute-based policies in ABAC capture more dynamic and context-dependent aspects of access control (Hu et al., 2014, p. 7). The analysis therefore considers both the technical behavior of the models and their implications for administrative workflows and organizational complexity.

3.5.1 Empirical Data Analysis

The empirical data collected from the Proof-of-Concept (PoC) artefact consists of structured observations of authorization behavior during the execution of predefined test scenarios. This data is generated automatically through the *MeasurementService* and reflects how each access control model processes authorization decisions in practice.
The analysis is primarily descriptive and comparative. Instead of applying statistical reasoning, the collected measurements are used to identify patterns in how RBAC and ABAC handle access decisions, resource visibility and policy evaluation. The collected metrics are

interpreted within the context of each model, rather than being treated as directly comparable quantitative variables.
The purpose of the empirical analysis is therefore not to generalize statistically, but to provide a structured and reproducible comparison of how the two models behave under identical conditions. This approach aligns with the experimental design of the study, where predefined scenarios act as controlled inputs for observing system behavior.

3.5.2 Structured Comparative Analysis

The evaluation of RBAC and ABAC follows a structured comparative approach based on clearly defined analytical dimensions, being granularity, administrative complexity and implementation complexity.
The comparison is performed across two cases (RBAC and ABAC) using identical test scenarios. This allows observed differences to be attributed to the underlying authorization models rather than variations in system configuration.
The analysis focuses on identifying systematic similarities and differences in how the models handle access control. Rather than isolating individual variables, each model is evaluated as a complete system, where configuration structures, policy logic and runtime behavior interact to produce authorization outcomes.
As emphasized by Ragin, (2014, p. 35), case-oriented comparison focuses on understanding how conditions within a case combine and interact to produce specific outcomes. This comparative approach aligns with case-oriented analysis, where the goal is to explain how different configurations of conditions across cases lead to different observed outcomes in practice (Ragin, 2014, p. 49).

3.5.3 Design Science Evaluation

Since the PoC constitutes a design science artefact, the analysis also follows the evaluation principles outlined by Hevner et al., (2004). The artefact is evaluated based on its ability to support the research objectives and to provide meaningful insights into the differences between RBAC and ABAC.
The evaluation focuses on three main aspects. First, *utility* by accessing whether the artefact enables meaningful comparison of authorization behavior. Second, *observed performance* by examining how each mode behaved under predefined scenarios. Lastly, *rigor*, ensured through controlled test execution, consistent measurement and iterative refinement of the artefact during development.

3.6 Validity and Reliability

Validity is about examining what one actually intends to examine, that is, the correspondence between the research question and the data collected (Patel & Davidsson, 2019, p. 129).
To ensure validity, the researcher must clearly define what is to be measured and then design an instrument that truly captures this, (Patel & Davidsson, 2019, pp. 129-130).
In this thesis, validity is supported by aligning the data collection directly with the research questions. The Proof-of-Concept (PoC) artefact was specifically designed to isolate authorization behavior, ensuring that observed differences between RBAC and ABAC can be attributed to the access-control models rather than external system factors.
The validity is strengthened by the clear definition of the evaluation dimensions in granularity, administrative complexity and implementation complexity, and ensuring that these dimensions are reflected in the collected measurements and scenario design. The use of

identical test scenarios for both models further ensures that comparisons are made under equivalent conditions.

Reliability refers to the consistency and reproducibility of the measurements (Patel & Davidsson, 2019, p. 131). In this study, reliability is ensured through controlled test execution, deterministic automated testing and standardized system configuration. Each scenario is executed under the same conditions, with the system state resetting between test runs to ensure consistent results.

In accordance with Hevner et al., (2004, pp. 87-88) DSR guideline 5, rigor is maintained through a clearly defined architecture, consistent measurement procedures and repeatable test scenarios. The use of automated testing ensures that data collection is systematic and not influenced by manual variation, supporting reproducibility of the results.

3.7 Limitations

This study has several limitations that should be considered when interpreting the results.
First, the PoC artefact are executed in a controlled environment with a limited dataset and a small number of users. This means that scalability aspects, such as performance and administrative overhead in large-scale deployments are not evaluated. Second. The study isolates RBAC and ABAC as separate models in order to enable controlled comparison. However, Microsoft Entra implements attribute-mechanisms as extension of RBAC. As a result, the findings reflect a conceptual comparison of the underlying access control models, rather than a direct evaluation of Entra's hybrid authorization approach.
Another limitation is that authentication was implemented using the Resource Owner Password Credentials flow (ROPC) to enable automated testing. While this improves reproducibility, it does not fully reflect modern production authentication practices, which may affect the considerations when evaluating the implementation choices.
Finally, implementation complexity is assessed based on structural and observed characteristics of the PoC artefact, rather than large-scale empirical data or statistical analysis. This means that conclusions regarding complexity are context-dependent and may not be applicable in different application environments.

3.8 Research Contributions

The study contributes to two of the three contribution categories defined by Hevner et al., (2004, p. 87) DSR guideline 4. The *design artefact contribution* is in the form of a functioning PoC system that enables controlled comparison of RBAC and ABAC within a Microsoft Entra-based environment. The artefact demonstrates how both models can be implemented and evaluated under identical conditions, providing practical insight into their behavior. The study also contributes *methodologically* with a structured evaluation approach that combines scenario-based testing, empirical observation and comparative analysis. This framework can be applied or extended in future research to evaluate access-control models in similar contexts.
The findings are presented in a way that addresses both technical and organizational perspectives. Technical audiences are provided with implementation details and architectural insights, while decision-makers are provided with an analysis of trade-offs between RBAC and ABAC in terms of flexibility, complexity and maintainability. This fulfills the DSR guideline 7 in Hevner et al., (2004, p. 90), which states that DSR outcomes must be communicated effectively to diverse audiences.

4. Implementation

This chapter presents the design and implementation of the Proof-of-Concept (PoC) artefact used to evaluate RBAC and ABAC within a Microsoft Entra-based environment. The purpose of the implementation is to create a controlled system in which both authorization models can be applied under identical conditions, enabling a systematic comparison of their behavior.
The chapter starts with an overview of the data model used in the implementation, including the project and user attributes used to represent access-control conditions. This is followed by a description of the authorization logic implemented for both ABAC and RBAC. The chapter then presents the test scenarios used in the evaluation, including their design, rationale and mapping to the research questions. Finally, the validation of the implementation is described to ensure correctness and reliability of the experimental setup.
A more detailed account of the technical design choices is provided in Appendix E.

4.1 System Overview

4.1.1 Project Attributes

The authorization scenarios in this study are based on a set of project resources (see Table 6). Each project is defined by attributes that are relevant to access-control decisions. These attributes serve as input to both ABAC policy evaluation and RBAC access constraints.
Each project contains the following attributes:

- **ProjectId** - unique identifier of the resource.
- **Department** - organizational domain to which the project belongs.
- **Status** - lifecycle state (“Active”/”Closed”).
- **IsSensitive** - indicates whether the project contains sensitive data.
- **Budget** - states the project's budget.
- **OwnerId** - identifier of the user/users that owns the project.

These attributes are used to model common access-control conditions, including sensitivity constraints, departmental separation, budget restriction policies, lifecycle-based constraints and ownership relationships between users and resources.

The dataset includes six projects (P1-P6), each designed to represent a specific combination of attributes such as high budget, sensitivity, inactivity or ownership (see Table 6).

Table 6. Project description and attributes.

ProjectId	Name	Description	Department	Status	OwnerId	Budget	IsSensitive	IntendedPurpose
P1	Internal Tools Portal	Baseline internal IT tools for daily operations	IT	Active	null	50000	FALSE	Baseline allow for viewers
P2	Sensitive Configuration Repository	Repository containing sensitive system configuration data	IT	Active	null	200000	TRUE	Scenario2 core sensitive project
P3	Legacy System Decommissioning	Decommissioning of legacy systems with sensitive data	IT	Closed	null	200000	TRUE	Inactive and sensitive gating
P4	High Budget Infrastructure Upgrade	Major IT upgrade exceeding standard budget limits	IT	Active	null	400000	FALSE	Budget constraint testing
P5	HR Compliance Records	Sensitive HR compliance documentation	HR	Active	null	100000	TRUE	Department mismatch testing
P6	Privileged Operations Project	Owned project overriding standard access rules	IT	Closed	U5	500000	TRUE	Scenario3 ownership override

4.1.2 User Attributes

Users in the system are defined through a set of attributes that represent both authorization-relevant properties (ABAC) and assigned roles (RBAC).
Each user includes:

- *ABAC attributes*:
  - **AccessLevel** - indicates whether the user can perform read or write operations.
  - **Department** - the organizational unit the user belongs to.
  - **CanViewSensitive** - boolean attribute controlling access to sensitive resources.
  - **CanViewInactiveProjects** - boolean attribute determining visibility of inactive resources.
  - **MaxVisibleProjectBudget** - defines the maximum budget a project can have, that the user is allowed to access.
  - **Projects** - list of projects owned by the user.
- *RBAC roles:*
  - **RBAC_Roles -** assigned roles used in RBAC authorization (*ProjectViewer, ProjectEditor, SensitiveViewer*).

Users are designed to represent distinct authorization profiles (see Table 7). This controlled variation enables systematic evaluation of how different authorization models handle combinations of attributes and role assignments.

Table 7. User attributes.

UserId	AccessLevel	Department	CanViewSensitive	CanViewInactiveProjects	Projects (Owner)	MaxVisibleProject Budget	RBAC_Roles	IntendedPurpose
U1	Viewer	IT	FALSE	FALSE	None	200000	ProjectViewer	Baseline constrained read-only viewer
U2	Viewer	IT	TRUE	FALSE	None	500000	ProjectViewer	Non-owner sensitive analyst (Scenario2 key user)
U3	Viewer	IT	TRUE	TRUE	None	500000	ProjectViewer	Inactive project read access testing
U4	Viewer	IT	TRUE	TRUE	None	100000	ProjectViewer	Budget-limited read-only viewer
U5	None	None	FALSE	FALSE	1, 2, 3, 4, 5, 6	0		Owner without attributes Scenario3 ownership override
U6	Viewer	IT, HR	TRUE	TRUE	None		ProjectViewer;SensitiveViewer	Superuser read reference
U7	Viewer, Editor	IT	TRUE	TRUE	None	500000	ProjectViewer;ProjectEditor	Non-sensitive editor
U8	Viewer, Editor	IT	TRUE	TRUE	None	500000	ProjectViewer;ProjectEditor;SensitiveViewer	Sensitive project editor

4.1.3 Attribute design rationale

The selection of attributes is grounded in access-control literature, particularly the ABAC framework defined by Hu et al., (2014, pp. 6-7), where authorization decisions are based on subject attributes, object attributes and contextual conditions.

The project attributes in this study (sensitivity, department, status, budget and ownership) correspond to object attributes commonly used to represent resource classification, organizational boundaries, lifecycle state and relationships between subjects and objects. Sensitivity attributes reflect classification-based controls similar to those found in Mandatory Access Control (MAC), while ownership attributes reflect relationship-based control as described in Discretionary Access Control (DAC) (Sandhu et al., 1996, p. 40).

User attributes such as department, access level and sensitivity permissions correspond to subject attributes describing organizational affiliation, capabilities and clearance level (Hu et al., 2014, pp. 7-9). Quantitative constraints such as budget limits further illustrate ABAC's ability to incorporate numeric conditions into policy evaluation (Hu et al., 2014, pp. 10, 29).

By combining classification, organizational, lifecycle, quantitative and relational attributes, the study models a set of access-control conditions that are both theoretically grounded and representative of common enterprise authorization requirements. This ensures that the evaluation scenarios reflect both theoretical ABAC constructs and realistic enterprise authorization requirements.

4.1.4 ABAC Authorization Logic

The ABAC implementation follows a resource-level, attribute–based decision model, where authorization is determined dynamically at request time through evaluation of subject attributes, object attributes and contextual conditions.

A user is granted read/write access to a project if it meets the criteria presented in Figure 1.

```
ALLOW if:
  Owner(user, project)
  OR
  (AccessLevel ∈ {Viewer, Editor}
   AND Department match
   AND Budget ≤ MaxVisibleBudget
   AND (IsSensitive → CanViewSensitive = true)
   AND (Status ≠ Active → CanViewInactiveProjects = true))
```

Figure 1. ABAC authorization logic.

All constraints must be satisfied simultaneously for access to be granted. The filtering is applied per resource, and ownership allows access regardless of other constraints.
Write operations follow the same visibility constraints as read access but additionally require the user to either own the resource or have the attribute "AccessLevel" = "Editor". This ensures that only authorized users can modify resources, while maintaining consistency with visibility rules. As a result, users cannot edit resources they are not permitted to view, unless they are the resource (project) owner.

4.1.5 RBAC Authorization Logic

The RBAC model in this study is intentionally implemented using a fixed role set without parameterization, reflecting the canonical RBAC model as defined in the literature. Where permissions are assigned to roles rather than dynamically evaluated at request time (Hu et al., 2014, pp. 5-6).
While it is possible to extend RBAC to support additional constraints through the introduction of more granular roles (e.g., roles supporting budget thresholds, project states or department access), such approaches are commonly associated with *role explosion*. Where the number of roles increases significantly as more contextual conditions are introduced (Hu et al., 2014, p. 6).

The system defines the following roles:

- **ProjectViewer** - allows read access to non-sensitive projects.
- **ProjectEditor** - allows modification of projects.
- **SensitiveViewer** - required for accessing sensitive projects.

Read access is governed by the formula described in
Figure 2, and write access is governed by the formula described in Figure 3:

```
ALLOW if:
  user has ProjectViewer role
AND
  if project is sensitive:
      user must also have SensitiveViewer role
```

Figure 2. RBAC Read access logic.

```
ALLOW if:
  user has ProjectEditor role
AND
  if project is sensitive:
      user must also have SensitiveViewer role
```

Figure 3. RBAC Write access logic.

Authorization is therefore based solely on role membership where decisions are binary (granted or denied). This model does not natively support ownership relationships, budget constraints or department-based filtering. Supporting equivalent context-dependent authorization constraints in RBAC would require the introduction of additional roles or role combinations. This would increase the total number of roles with the associated administrative overhead. The RBAC implementation therefore represents a baseline model that reflects common role-based access control patterns, rather than an extended or highly specialized configuration.
The implemented models reflect fundamental differences in access-control semantics, displayed in Table 8.

Table 8. Differences in access-control semantics.

Aspect	**ABAC**	**RBAC**
Decision basis	Attributes (subject + resource + context)	Roles
Granularity	Fine-grained (per resource)	Coarse-grained
Overrides	Supported (ownership override)	Not natively supported
Adaptability	Dynamic (attribute changes affect access)	Static (requires role updates)
Complexity	Policy-based	Role-assignment based

4.2 Test Scenarios

This section presents the four authorization scenarios used in this study, as well as motivations for their selection based in literature. Each scenario is derived from common access-control use cases and is motivated by established access-control literature. The scenarios are designed to expose differences in policy expressiveness, decision semantics and administrative characteristics between ABAC and RBAC.

The scenarios are intended to support qualitative comparisons of authorization behavior, administrative characteristics and policy evolution, not quantitative performance or scalability evaluations. Each scenario is designed to isolate specific access-control characteristics in order to support structured comparison across the research questions.

4.2.1 Scenario1 - Read-heavy project listing with filtered visibility

This scenario evaluates read-heavy access patterns by examining how users with different authorization profiles retrieve a list of projects through repeated *GET /projects* requests. The scenario focuses on visibility semantics rather than request-level denial, enabling a direct comparison of how RBAC and ABAC determine which resources are visible to an authorized user. In the ABAC implementation, access decisions are made at the resource level, meaning that individual projects may be included or excluded based on evaluated attributes. In contrast, the RBAC implementation applies authorization primarily at the role level, resulting in binary inclusions/exclusion determined by role membership.

The Yahoo! Cloud Serving Benchmark (YCSB) is a commonly used framework for simulating real-world workloads and explicitly models read-heavy, write-heavy and balanced scenarios, with read-heavy being the baseline workload (Liyanage et al., 2025, pp. 7-8). RBAC-focused benchmarking literature similarly identifies *SELECT-based* queries as among the most common operations in data management systems and therefore a natural starting point when evaluating authorization behavior (Hansen et al., 2026, p. 26).
In this thesis, the proof-of-concept implements an API, to issue SQL queries against a SQLite database for both the ABAC and RBAC routes. When an authenticated user invokes *GET /projects*, the application retrieves all project records from the database and then applies authorization logic to determine which projects are visible in the response. This mirrors the access-list and read-heavy workload pattern described in the literature and it provides a suitable foundation for comparing authorization models.

The purpose of Scenario1 is to demonstrate differences in authorization granularity and flexibility (RQ1). ABAC enables partial result sets through attribute evaluation, while RBAC produces uniform result sets for users with identical roles. The scenario also provides supporting insight into administrative workflow differences (RQ2), as RBAC relies on role assignments to manage read access whereas ABAC relies on attribute configuration to express operational constraints such as sensitivity, project status and budget thresholds.
The *GET /projects* endpoint was invoked for a set of users selected to represent distinct authorization profiles (see Table 9). Each selected user executed the request once against the ABAC implementation, and once against the RBAC implementation. This approach ensured that all relevant authorization paths were exercised while avoiding redundant executions that did not contribute additional analytical insight.

Table 9. Scenario1 user categories.

Category	User	Roles	Key Attributes	Purpose
A - Constrained viewer	U1	ProjectViewer	CanViewSensitive = false, CanViewInactiveProjects = false, MaxVisibleProjectBudget = 200 000, Department = IT	Demonstrates strong ABAC filtering while retaining baseline RBAC visibility
B - Inactive-allowed viewer	U3	ProjectViewer	CanViewSensitive = true, CanViewInactiveProjects = true, Department = IT	Demonstrates attribute-based authorization without corresponding RBAC role assignment ABAC
C - Budget-limited viewer	U4	ProjectViewer	MaxVisibleProjectBudget = 100 000, CanViewSensitive = true, CanViewInactiveProjects = true	Demonstrates budget-based ABAC constraints, ignored by RBAC
D - Full-privilege viewer	U6	ProjectViewer, SensitiveViewer	Unrestricted attributes, multi-department access	Validates upper-bound visibility in both models

User U3 is intentionally configured with attribute-based sensitivity permission (*CanViewSensitive*= true) without assigning the corresponding RBAC role. This does not reflect a limitation in RBAC's capability to grant access but rather highlights a fundamental difference in how permissions are expressed and managed. In RBAC, equivalent access would require explicit assignment of the *SensitiveViewer* role, whereas in ABAC the same outcome is achieved through attribute evaluation without modifying the role membership.

All users in Scenario1 were authorized at the endpoint level, allowing the analysis to focus exclusively on differences in result-set visibility, rather than request denial.

4.2.2 Scenario2 - Sensitive resource access control

This scenario evaluates authorization behavior when users attempt to access a single sensitive resource using the *GET /projects/{id}* endpoint. The objective is to isolate how resource sensitivity is enforced under ABAC and RBAC. A project marked as sensitive (*IsSensitive*= true) was selected, and access attempts were performed using users representing different sensitivity authorization profiles (see Table 10). Unlike Scenario1, which focuses on result-set filtering, this scenario evaluates binary access decisions at the resource level.

Under ABAC, authorization decisions are evaluated based on a combination of subject attributes, object attributes and contextual conditions (Hu et al., 2014, pp. 6-7). This allows policies to explicitly reference resource sensitivity. In contrast, RBAC determines access by evaluating whether the subject is assigned to a role authorized for the request operation (Hu et al., 2014, pp. 5-6; Sandhu et al., 1996, pp. 38-39). Hu et al., (2014, p. 6) further noted that RBAC has limited native support for multi-factor authorization decisions. Trying to support this often leads to the creation of numerous ad-hoc rules, a phenomenon commonly referred to as "role explosion" (Hu et al., 2014, pp. 6).

The scenario is designed to demonstrate differences in authorization granularity and flexibility (RQ1), as ABAC enables access through attribute evaluation without requiring predefined role assignments, whereas RBAC expresses equivalent permissions through role assignments. It further supports analysis of administrative workflow differences (RQ2), as RBAC requires explicit role assignments to manage sensitivity, whereas ABAC relies on attribute configuration. Although sensitivity is the primary focus of this scenario, ABAC policies evaluate multiple attributes conjunctively. Even when a user satisfies the sensitivity condition, access may still be denied due to other constraints such as department mismatch, as demonstrated in scenario2 (S2-T7, see Appendix D, *Scenario2 Results (Id=7)*).

Table 10. Scenario2 user categories.

Category	**User**	**Roles**	**Attributes**	**Purpose**
A - No sensitivity access	U1	ProjectViewer	CanViewSensitive=false	Denied in both models
B -Attribute-based sensitivity access	U2	ProjectViewer	CanViewSensitive=true	Allowed in ABAC, Denied in RBAC
C - Fully authorized user	U6	ProjectViewer, SensitiveViewer	CanViewSensitive=true	Allowed in both models

User U2 is intentionally assigned attribute-based sensitivity permission without being granted the corresponding RBAC role. This reflects a difference in how access is expressed rather than a limitation in RBAC's ability to grant such access, since it is clearly displayed for user U6. Rather, it highlights a difference in how access is expressed. In RBAC, equivalent access would require explicit assignment of the *SensitiveViewer* role, whereas in ABAC, access can be derived dynamically from user attributes without modifying role membership.

For each user, the request was executed once against both the ABAC and RBAC implementations. This ensured direct comparability between attribute-based and role-based sensitivity enforcement while avoiding redundant executions.

4.2.3 Scenario3 - Ownership-based access overrides

This scenario evaluates how access-control models handle ownership-based overrides, where a user is granted access to a resource despite failing standard authorization constraints. The objective is to examine differences in how RBAC and ABAC support context-dependent authorization decisions.
A project with an assigned owner was selected, and access attempts were performed using both the owning user and non-owning users (see Table 11). The selected user intentionally does not satisfy standard access conditions such as sensitivity permissions, department alignment, budget constraints or project status requirements.
Under ABAC, ownership is represented as an object attribute, allowing policies to grant access based on ownership independently of other constraints. As a result, access may be granted even when other attribute conditions are not satisfied. In contrast, RBAC evaluates access exclusively through role membership, and does not natively support ownership-specific overrides without introducing additional roles or external logic.

The scenario is designed to demonstrate differences in authorization flexibility (RQ1), particularly in the ability to express special cases. It also highlights differences in implementation complexity and administrative workflow (RQ2), as RBAC would require additional role assignments to model ownership-specific access. Finally, the scenario provides direct insight into policy maintenance and change management (RQ3), since ownership-based access in ABAC is automatically updated through changes to object attributes, whereas RBAC requires explicit changes to role assignments.

Table 11. Scenario3 user categories.

Category	User	Roles	Attributes	Purpose

A - Ownership override	U5	None	Does not satisfy standard access constraints	Demonstrates ABAC ownership-based overrides
B - Non-owner comparison	U1	ProjectViewer	Fails multiple attribute constraints	Confirms absence of override behavior if not owner

For each user, the *GET /projects/{id}* endpoint was invoked against both the ABAC and RBAC implementations. This allowed direct comparison of ownership-based override handling across the two models.

This scenario intentionally evaluated a class of authorization requirements that are not natively supported in the RBAC model. Ownership-based access can be modeled in ABAC by defining policies that evaluate both subject and object attributes, allowing access decisions to be based on relationships between users and resources, rather than solely on predefined roles (Hu et al., 2014, pp. 5-6, 10).
The inclusion of this scenario is therefore not intended to demonstrate a deficiency in the RBAC implementation, but to examine how each model handles context-dependent access requirements at a conceptual level. In RBAC, such behavior can be approximated through additional roles or external logic, but this introduces increased complexity and administrative overhead. Such approaches are not modeled in this study in order to preserve a clear distinction between role-based and attribute-based authorization mechanisms.

By contrast, ABAC can express ownership directly as an object attribute within policy evaluation. To validate that ownership alone is responsible for the observed authorization behavior, both an owner and a non-owner user were evaluated under identical conditions. This ensured that access decisions could be attributed specifically to ownership rather than other policy attributes.

The scenario evaluated whether a user may gain access to a project based solely on ownership, even when other authorization constraints are not satisfied.
Hu et al., (2014, p. 9) explicitly models ownership as an object attribute within ABAC. They also describe how access-control policies are written by evaluating subject attributes against object attributes (Hu et al., 2014, pp. 9-10). Because ownership is represented as attributes rather than a predefined role, ownership-based access can be expressed directly in policy without modifying role assignments.
RBAC by contrast, evaluates access primarily based on role membership (Hu et al., 2014, pp. 5-6; Sandhu et al., 1996, pp. 38-39). The authors note that RBAC does not easily support contextual or exception-based authorization requirements and that accommodating such cases often requires additional roles or manual role management, increasing administrative overhead and complexity (Hu et al., 2014, pp. 5-6).

4.2.4 Scenario4 - Write operations (create, update, delete)

Hu et al., (2014) defined the purpose of logical access control as protecting objects from unauthorized operations, listing “creating, editing and deleting” among the supported operations (Hu et al., 2014, p. 4). This establishes write operations as a core responsibility for access-control systems. Furthermore, the authors state that every object within a system must have at least one policy defining the access rules for allowable subjects and allowable operations (Hu et al., 2014, p. 10). As a result, write operations on protected objects are within the scope of access-control policy evaluation.

ABAC supports expressive write authorization by evaluating subject attributes, object attributes and contextual decisions at decision time (Hu et al., 2014, pp. 6-7). In contrast, RBAC grants write access structurally through role membership rather than through dynamic attribute evaluation (Hu et al., 2014, pp. 5-6; Sandhu et al., 1996, pp. 38-39).

This scenario evaluates authorization behavior for modification, including update, create and delete operations of project resources. Users with different permission statuses are selected to show differences in write access, including sensitive resources (see Table 12). The purpose is to examine how RBAC and ABAC enforce access control for write actions, and how authorization logic differs when multiple constraints are involved. Unlike previous scenarios that focus on read access, write operations introduce additional requirements, including editor permissions and resource-level visibility. In both models, users must satisfy write permissions, and for update/delete also have visibility over the target resource.

RBAC provides a clear and structured mechanism for managing write permissions through role assignments. In the RBAC implementation it requires users to be assigned the *ProjectEditor* role to gain write access over projects. Additional roles, such as *SensitiveViewer*, must be combined to grant access to sensitive resources. This results in a structural authorization model where permissions are defined through role combinations.

In contrast ABAC evaluates write authorization using the same attribute-based policies as read access, including sensitivity, department and contextual constraints. This enables fine-grained control over write permissions without required predefined role combinations.
The scenario is designed to demonstrate differences in authorization expressiveness and flexibility (RQ1), as well as differences in implementation complexity and administrative workflow (RQ2). Additionally, it provides insight into policy maintenance and change management (RQ3), as changes to resource attributes can alter access decisions dynamically in ABAC, whereas RBAC requires role reassignments.

Table 12. Scenario4 user categories.

Category	User	Roles	Attributes	Purpose
A - Editor without sensitivity role	U7	ProjectViewer, ProjectEditor	AccessLevel=Viewer, Editor CanViewSensitive=True	Tests sensitivity restrictions
B - Full editor	U8	ProjectViewer, ProjectEditor, SensitiveViewer	AccessLevel=Viewer, Editor CanViewSensitive=True	Confirms full write capabilities
C - Non-editor	U1	ProjectViewer	AccessLevel=Viewer	Negative control for write permissions

User U7 is intentionally configured with attribute-based sensitivity permission (*CanViewSensitive*= true) without assignment of the corresponding RBAC role *SensitiveViewer*. This does not indicate a limitation in RBAC's ability to grant such access. Rather, it highlights a difference in how permissions are expressed. In RBAC, equivalent access would require combining the *ProjectEditor* and *SensitiveViewer* roles, through explicit role assignment. In ABAC, the same decision is evaluated at request time based on user and resource attributes. As a result, changes in resource characteristics (such as sensitivity) are

reflected immediately in authorization decisions without requiring reconfiguration of role assignments, although attribute values themselves must still be managed administratively.
For each user, write operations were performed on both sensitive and non-sensitive projects using the *PUT /projects/{id}* endpoint. Each request was executed once against the ABAC and RBAC implementations to enable direct comparison of authorization behavior while avoiding redundant executions.

In Scenario4, certain update operations modify the sensitivity of resources (S4-T1, see Appendix D, *Scenario4 Results (Id=1)*), which alters the authorization requirements for subsequent requests (S4-T2, see Appendix D, *Scenario4 Results (Id=1)*). This design illustrates how changes in resource characteristics may require role reassessment in RBAC, whereas ABAC policies adapt dynamically through attribute evaluation. Additional requests were executed where user U7 was tested against *POST /projects* for both the RBAC and ABAC implementations, as well as testing the *DELETE /projects* endpoint for both models. This allowed all write endpoints to be tested, providing additional insight to both endpoint access with the *POST* request, as well as dynamic evaluation based on contextual attributes with the *PUT* and *DELETE* requests.

4.3 Validation of Authorization Implementation

Before conducting the scenario-based evaluation, a comprehensive validation test suite was executed to ensure that both the ABAC and RBAC implementations correctly enforced their respective authorization rules, the design of the validation test suite can be found in Appendix C. The purpose of the validation tests was to verify that all defined policies behaved as expected across all supported endpoints and policy conditions.
The validation suite covered all endpoints used in the scenarios and covered all ABAC attribute conditions, including sensitivity, department constraints, project status, budget limits and ownership overrides. It also covered all RBAC role combinations for *ProjectViewer*, *ProjectEditor* and *SensitiveViewer*. Each validation test was designed to isolate a specific authorization condition, ensuring that both positive and negative cases were evaluated. The tests were executed against a controlled dataset, and results were recorded using the system's *MeasurementService* logs. The results of the validation test can be found in Appendix C.

4.3.1 ABAC Validation

Attribute-based filtering behaved as expected, producing partial result sets based on combinations of sensitivity, department, budget and project status constraints. Multi-attribute evaluation was verified, where access was denied if any required condition failed. Ownership override was correctly enforced, allowing access even when all other attribute constraints were not satisfied. Write operations (*POST*, *PUT*, *DELETE*) were correctly restricted based on the attribute *AccessLevel*, with only users possessing the attribute value "Editor" were permitted to perform modifications.

4.3.2 RBAC Validation

Read access was correctly governed by the *ProjectViewer* role. Sensitive resource access required the *SensitiveViewer* role as intended. Write operations were restricted to users with the *ProjectEditor* role. Combined permissions for updating sensitive resources, did require multiple roles (*ProjectEditor* + *SensitiveViewer*), confirming role composition behavior.

4.3.3 Validation Results Conclusion

The validation results demonstrate that both the ABAC and RBAC implementations behave as intended and correctly enforce their respective authorization policies. This ensures that the subsequent scenario-based evaluation reflects true differences between the models, rather than implementation errors or inconsistencies.

5. Results

The results presented in this chapter summarize the observable authorization behavior of RBAC and ABAC across the defined scenarios. The focus is on metrics that are directly comparable between the two models and that reflect authorization logic rather than performance characteristics.

The primary metrics used in the analysis are *AccessGranted, VisibleResourceCount* and *DecisionCount*. These provide consistent indicators of authorization outcomes, user-visible access and the level of evaluation performed during each request. Additional metrics are included in the full dataset to support interpretation (See Appendix D), but are not used as primary indicators for cross-model comparison.
For each scenario, summarized results are presented to highlight key differences in how RBAC and ABAC evaluate and enforce access control under identical conditions. Detailed measurements and full test outputs are available in Appendix D.

5.1 Scenario1

The design of the test suite and full results are presented in Appendix D. The summarized results are shown in Figure 4.

User profile	Model	AccessGranted	VisibleResourceCount	DecisionCount	Key Characteristics
Constrained viewer (U1)	ABAC	Yes	1	7	Strong filtering across sensitivity, budget, inactive status
Constrained viewer (U1)	RBAC	Yes	2	1	Binary inclusion based on role, sensitive resources excluded
Inactive-allowed viewer (U3)	ABAC	Yes	5	7	Attribute-based inclusion of inactive resources and expanded visibility
Inactive-allowed viewer (U3)	RBAC	Yes	2	1	Identical visibility to U1 due to unchanged role configuration
Budget-limited viewer (U4)	ABAC	Yes	1	7	Budget constraint limits visibility; high-budget resources excluded
Budget-limited viewer (U4)	RBAC	Yes	2	1	Budget constraints not enforced, visibility identical to other viewers
Full-privilege viewer (U6)	ABAC	Yes	6	7	Full visibility achieved through satisfaction of all attribute constraints
Full-privilege viewer (U6)	RBAC	Yes	6	1	Full visibility achieved through combined roles (ProjectViewer + SensitiveViewer)

Figure 4. Scenario1 summarized test results.

From these results, several observations can be made.

ABAC per-resource filtering:
The ABAC results show that different users observe different subsets of the project list. Although the number of evaluated resources (DecisionCount = 7) is consistent across users, the number of visible resources varies. Individual projects are included or excluded based on evaluated attribute conditions, resulting in partial result sets for different users.

RBAC role-based visibility:
In contrast, RBAC results show identical visibility for users with the same role configuration. Users U1, U3 and U4 all observe the same number of projects, despite differences in attributes such as budget limits and access to inactive resources. Visibility changes only when the assigned role set changes, as demonstrated by user U6, who observes all projects due to the addition of the *SensitiveViewer* role.

Decision structure differences:
The ABAC evaluation outputs include multiple denial reasons per request (captured in the *DenialReasonSummaryJson* variable) in the measurement logs (see Figure D2 in Appendix D). This along with higher values for *DecisionCount* indicates that multiple policy elements are evaluated for each resource.
In RBAC, decisions are based on a single role check, resulting in lower *DecisionCount* values and single-point denial reason per request.

Evaluation scope differences:
The results indicate that RBAC evaluates authorization once per request, while ABAC performs evaluation for each resource.

- RBAC: DecisionCount = 1
- ABAC: DecisionCount = 7

This indicates that the number of evaluated conditions differs between the models, with ABAC evaluating multiple conditions across multiple resources, while RBAC evaluates a single role-based condition per request.

5.2 Scenario2

The design of the test suite and full results for Scenario2 are presented in Appendix D. The summarized results are shown in Figure 5.

User profile	Model	AccessGranted	VisibleResourceCount	DecisionCount	Key Characteristics
Constrained viewer (U1)	ABAC	No	0	2	Sensitive access denied due to missing attribute (CanViewSensitive)
Constrained viewer (U1)	RBAC	No	0	2	Sensitive access denied due to missing role (SensitiveViewer)
Attribute-based viewer (U2)	ABAC	Yes	1	2	Sensitive access allowed through attribute (CanViewSensitive = true)
Attribute-based viewer (U2)	RBAC	No	0	2	Access denied due to absence of SensitiveViewer role
Full-privilege viewer (U6)	ABAC	Yes	1	2	Access granted; all attribute constraints satisfied
Full-privilege viewer (U6)	RBAC	Yes	1	2	Access granted due to combined roles (ProjectViewer + SensitiveViewer)
Attribute-based viewer (U2, mismatch case)	ABAC	No	0	2	Multi-attribute evaluation: department mismatch overrides sensitivity permission

Figure 5. Scenario2 summarized test results.

This scenario evaluates authorization behavior when accessing a single sensitive resource, enabling comparison between attribute-based and role-based enforcement.
From the results, several observations can be made.

Attribute/based access to sensitive resources (ABAC):
The ABAC results show that access to sensitive resources depends on the evaluation of user attributes. For the constrained viewer (user U1), access is denied due to the absence of the *CanViewSensitive* attribute. As a result, the user is unable to access the resource, and the request is rejected before inclusion in the result set.
In contrast, user U2 is granted access, as the attribute *CanViewSensitive*= true satisfies the sensitivity requirement.

Conjunctive evaluation of multiple attributes:
The results show that ABAC evaluates multiple attributes simultaneously.
Although user U2 satisfies the sensitivity condition *CanViewSensitive*= true, access is denied when the requested project belongs to a different department. This indicates that multiple attribute conditions are evaluated together when determining access.

Role-based access (RBAC):
The RBAC results show that access to sensitive resources depends on role membership. Users without the *SensitiveViewer* role (U2) are denied access to sensitive resources, while users with the role assigned (U6) gains access.

5.3 Scenario3

The design of the test suite and full results for Scenario3 are presented in Appendix D. The summarized results are shown in Figure 6.

User profile	Model	AccessGranted	VisibleResourceCount	DecisionCount	Key Characteristics
Owner without attributes (U5)	ABAC	Yes	1	1	Ownership override grants access despite failing all attribute constraints
Owner without attributes (U5)	RBAC	No	0	1	Access denied due to absence of required roles; ownership not represented in RBAC
Non-owner constrained user (U1)	ABAC	No	0	2	Denied after multi-attribute evaluation (sensitivity, inactive status, budget)
Non-owner constrained user (U1)	RBAC	No	0	2	Denied due to missing role and lack of sensitivity permission

Figure 6. Scenario3 summarized test results.

This scenario evaluates how the two authorization models handle ownership-based access when a user does not satisfy standard authorization constraints. From the result, several observations can be made.

Ownership-based access in ABAC:
The ABAC results show that access can be granted based on ownership, even when other attribute conditions are not satisfied.
The user U5, who does not satisfy standard access conditions such as sensitivity permissions, department alignment, budget constraints or project status requirements, is still granted access to the resource.
This access is associated with a single authorization decision ("DecisionCount" = 1), indicating that the ownership condition short-circuits further evaluation.

Ownership and access outcome in RBAC:
In contrast, the RBAC results show that user U5 is denied access to the resource.
Since ownership is not represented within the RBAC role assignments, access decisions are based solely on role membership. In this scenario, the absence of required roles results in a denied request.

Comparison of access outcomes:
The results show that access decisions differ between ABAC and RBAC when ownership is the only condition satisfied.
For the same request:

- ABAC grants access when the ownership condition is satisfied.
- RBAC denies access when required roles are not assigned.

Control case comparison:
For non-owning users (e.g., user U1), access is denied in both models when standard authorization conditions are not satisfied.
This indicates that ownership is the differentiating factor in cases where access is granted under ABAC but denied under RBAC in this scenario.

5.4 Scenario4

The design of the test suite and full result for Scenario4 are presented in Appendix D. The summarized results are shown in Figure 7.

User profile	Model	AccessGranted	VisibleResourceCount	DecisionCount	Key Characteristics
Editor without sensitivity role (U7)	ABAC	Yes	2	2	Allowed to update both non-sensitive and sensitive resources based on attribute evaluation (CanViewSensitive = true)
Editor without sensitivity role (U7)	RBAC	No	1	2	Denied when resource becomes sensitive due to missing SensitiveViewer role; requires role combination
Non-editor viewer (U1)	ABAC	No	1	2	Denied due to failing write requirements and sensitivity constraints
Non-editor viewer (U1)	RBAC	No	0	1	Denied due to missing ProjectEditor role
Full-privilege editor (U8)	ABAC	Yes	2	2	Full access as all attributes satisfy policy constraints
Full-privilege editor (U8)	RBAC	Yes	2	2	Full access enabled through combined roles (ProjectEditor + SensitiveViewer)
Editor without sensitivity role (U7 – create)	ABAC	Yes	1	1	Create operation allowed via attribute (AccessLevel = Editor)
Editor without sensitivity role (U7 – create)	RBAC	Yes	1	1	Create operation allowed via role (ProjectEditor)
Editor without sensitivity role (U7 – delete)	ABAC	Yes	2	2	Sensitive delete allowed through attribute evaluation
Editor without sensitivity role (U7 – delete)	RBAC	No	1	2	Sensitive delete denied due to missing SensitiveViewer role

Figure 7. Scenario4 summarized test results.

This scenario evaluates authorization behavior for write operations, including update (*PUT*), create (*POST*) and delete (*DELETE*) requests, under both RBAC and ABAC. From the results, several observations can be made.

Attribute-based write authorization (ABAC):
The ABAC results show that write authorization is determined through attribute evaluation.
For user U7, write operations on both non-sensitive and sensitive resources are permitted when the attribute *CanViewSensitive*=true and the user has the required *AccessLevel*="Editor" attribute value. For user U8, who satisfies all relevant attribute conditions, write operations are permitted across all tested endpoints.

Combined role requirements:
The results show that write access to sensitive resources under RBAC requires a combination of roles. Access to non-sensitive resources is granted with the *ProjectEditor* role, while access to sensitive resources requires the user to also have the role *SensitiveViewer*.

Changes in resource attributes:
The results show that modifications to resource attributes affect subsequent authorization decisions.
When a resource is updated and marked as sensitive by user U7:

- Subsequent RBAC requests require the *SensitiveViewer* role for access
- ABAC evaluation reflects the updated resource attribute during authorization and requires the *CanViewSensitive*=true attribute value.

Comparison of access outcomes:
The results show that access decisions differ between ABAC and RBAC when multiple conditions, such as sensitivity and access level, are involved.
For equivalent requests, ABAC grants or denies access based on attribute evaluation while RBAC access decisions are based on role membership and role combinations.

6. Analysis

This chapter analyzes the results presented in Chapter 5 in relation to the theoretical foundations discussed in Chapter 2. The purpose is not only to describe observed differences between RBAC and ABAC, but to interpret these differences in terms of authorization behavior, system design and administrative implications.
The analysis follows a structured, research-question driven approach. Each section addresses one of the research questions by linking empirical observations from the defined scenarios to concepts established in the literature. Rather than treating results as isolated measurements, the analysis focuses on explaining how the underlying characteristics of each model give rise to the observed authorizations outcomes.

The interpretation is based on the primary comparative metrics introduced in Chapter 3 (*MeasurementService)*, combined with qualitative insights from the scenario results. This enables a systematic evaluation of how RBAC and ABAC differ in terms of granularity, flexibility, implementation complexity and policy management.

In addition, the analysis is informed by a sociotechnical perspective, recognizing that access control models do not operate solely as technical mechanisms, but reflect broader organizational structures, responsibilities and governance practices. The findings are therefore interpreted both as differences in authorization logic and as differences in how access control is structured and managed in practice.

6.1 RQ1 - Authorization Granularity and Flexibility

The literature describes ABAC as a fine-grained authorization model, where access decisions are based on multiple attributes of the subject, object and environment (Hu et al., pp. 5-7). Whereas RBAC is typically characterized as a coarser-grained model, where access decisions are derived from role membership rather than evaluated per resource (Sandhu et al., 1996, pp. 38-40; Hu et al., 2014, pp. 5-6).
The empirical results align with and extend this distinction across all evaluated scenarios.

6.1.1 Per-resource vs role-based decision semantics

Scenario1 demonstrates that ABAC performs per-resource authorization evaluation. Producing partial result sets where individual resources are included or excluded based on attribute evaluation. Users with different attribute combinations observe different subsets of resources, even when invoking the same endpoint.

In contrast, RBAC produces uniform visibility for users with identical role assignments. Access is determined at the role level rather than at the level of individual resources, resulting in identical result sets regardless of contextual differences.
This supports the characterization of ABAC as a fine-grained model and RBAC as a coarse-grained model, where granularity is tied to the level at which access decisions are evaluated.

6.1.2 Conjunctive policy evaluation

The results also extend the literature by demonstrating that ABAC evaluates multiple attributes simultaneously as part of a combined policy rule.
In scenario2 (S2-T7, see Appendix D, *Scenario2 Results (Id=7)*), access is denied despite satisfying the sensitivity condition because another constraint (department mismatch) is violated. This illustrates that ABAC does not evaluate attributes in isolation, but rather as part of a combined policy rule.

This behavior aligns with the ABAC model described by Hu et al. (2014, pp. 5-7, 10), where authorization decisions are derived from boolean policy rules combining subject, object and environmental attributes. The empirical results therefore provide a concrete demonstration of these theoretical characteristics.
In contrast, RBAC authorization is based on role membership and does not natively support combining multiple contextual constraints within a single decision (Hu et al., 2014, pp. 5-6). Supporting equivalent behavior requires introducing additional roles, which increases structural complexity (Hu et al., 2014, pp. 5-6).

6.1.3 Context-dependent and exception-based authorization

Scenario3 further extends the analysis by demonstrating the ability to express context-dependent exceptions. ABAC supports ownership-based access through attribute evaluation, allowing access decisions to be influenced by relationships between users and resources.

In contrast, RBAC does not natively represent ownership as part of the authorization model, and equivalent behavior requires introducing additional roles or application-level logic.
Scenario4 reinforces these findings in a write-operation context, where ABAC evaluates multiple conditions dynamically across different endpoints, while RBAC relies on predefined role combinations.

6.1.4 Conclusions for RQ1

The empirical results show that:

- ABAC provides fine-grained, context-sensitive authorization based on per-resource evaluation.
- RBAC provides coarse-grained, role-based authorization based on role membership.
- ABAC supports multi-attribute and conjunctive decision logic evaluated at request time.
- RBAC relies on static role definitions and role combinations to express access rules.

Taken together, the results indicate that ABAC offers greater expressiveness and flexibility in representing complex and context-dependent authorization requirements in the evaluated scenarios.

6.2 RQ2 - Implementation and Administrative Complexity

The literature presents a mixed view on complexity. RBAC simplifies access control administration by assigning permissions to predefined roles based on organizational job functions, reducing the need to manage permissions at the individual user level (Sandhu et al.,

1996, p. 38). In contrast, ABAC is more complex to implement and maintain, as it evaluates access decisions based on multiple attributes and supports a large number of possible policy combinations (Hu et al., 2014, p. 33). The empirical findings both support and refine this perspective.

6.2.1 Implementation complexity in practice

From the PoC implementation some observations can be made.

- RBAC required minimal application logic, primarily relying on role checks.
- ABAC required explicit policy definitions, requirement handlers and attribute retrieval.

This confirms the literature's claim that ABAC introduces greater implementation complexity at the code level (Hu et al., 2014, p. 33). However, the results also show complexity manifests differently across the models.
Scenario2 illustrates that RBAC requires explicit role assignments (e.g., *SensitiveViewer*) to represent access conditions that in ABAC, are expressed through attribute evaluation. This reflects the role-centric structure described by Sandhu et al. (1996, pp. 38-40), where permissions are tied to roles rather than evaluated dynamically.

Scenario4 further demonstrates that RBAC requires combinations of roles (*ProjectEditor* + *SensitiveViewer)* to represent a single logical capability. This demonstrates that RBAC complexity emerges not only during initial implementation, but during policy extension and composition, supporting the concept of "role explosion", where increasing contextual requirements lead to a proliferation of roles (Hu et al., 2014, pp. 5-6; Sandhu et al., 1996, pp. 45-46). It should be noted that RBAC provides a highly structured and predictable authorization model, which simplifies implementation and auditing in environments with stable access requirements.

6.2.2 Structural vs policy-based complexity

The results indicate that RBAC complexity increases through structural expansion. As additional conditions are introduced, new roles or role combinations are required. In contrast, ABAC concentrates complexity within policy definitions and attribute management. Increasing the number of conditions would therefore naturally lead to more complex policy definitions and more expansive attribute management requirements.

Scenario3 demonstrates the differences clearly. Ownership-based access is expressed in ABAC through a single attribute condition (*Projects/OwnerId*), whereas RBAC requires additional modeling effort, such as introducing roles or application-level logic.

6.2.3 Conclusion for RQ2

The results indicate that:

- RBAC has lower initial implementation complexity.
- RBAC complexity increases through role composition and structural growth.
- ABAC introduces more complex policy logic and attribute management.
- ABAC centralizes complexity within policies, rather than distributing it across role structures.

This suggests that complexity is not simply higher in ABAC, but differently structured between the models. RBAC offers structural clarity, while ABAC provides flexibility through centralized policy design.

6.3 RQ3 - Policy Maintenance and Adaptability

The literature emphasizes that ABAC enables dynamic adaptation of access control decisions through attribute-based evaluation, where changes in attributes can be reflected without modifying policies (Hu et al., 2014, pp. 10-11). In contrast, RBAC relies on administrative actions such as assigning or revoking user membership in roles to reflect changes in access needs (Sandhu et al., 1996, pp. 38-39). The empirical results support this distinction.

6.3.1 Dynamic adaptation to attribute changes

Scenario4 shows that changes to resource attributes directly affect authorization outcomes under ABAC. When a resource is updated and marked as sensitive, subsequent authorization decisions reflect the updated attribute without requiring changes to policy definitions. This demonstrates that ABAC policies are evaluated at request time and incorporate current attribute values into access decisions.
In contrast, RBAC requires an update to role assignment or role combinations to reflect changes in access requirements. When a resource becomes sensitive, users must be assigned additional roles (*SensitiveViewer*) to maintain access.
Scenario3 further supports this observation. Ownership-based access is automatically reflected in ABAC through attribute evaluation, while RBAC requires explicit changes to role assignments or structural modifications to represent equivalent relationships. Modifying a resource's attributes, setting (*IsSensitive*= true) immediately affects authorization decisions under ABAC without requiring any policy changes.
This reflects the core principle of ABAC described by Hu et al. (2014, pp. 10-11), where authorization decisions are evaluated at request time using current subject and object attribute values with policy rules. The empirical results demonstrate this behavior in practice, as changes to resource attributes directly influence access decisions without requiring modifications to policy definitions.

In contrast, RBAC relies on role assignments to determine access (Sandhu et al., 1996, pp. 38-40). As a result, changes in access requirements must be reflected through updates to role assignments or role structures. This difference highlights the distinction between dynamic attribute-based evaluation and static role-based authorization.
When a resource becomes sensitive:

- ABAC enforces the updated condition directly through attribute evaluation.
- RBAC requires users to be assigned additional roles ("SensitiveViewer").

This demonstrates that RBAC policies are structurally static, requiring administrative intervention for adaptation. That supports the literature's view that ABAC is well suited to environments where access conditions change, as authorization decisions can adapt dynamically through changes in attribute values rather than policy updates (Hu et al., 2014, pp. 10-11).

6.3.2 Administrative overhead over time

The results suggest that:

- ABAC reduces the need for structural updates in dynamic environments but shifts complexity toward attribute management and policy maintenance.
- RBAC increases long-term maintenance effort due to role updates and management.

6.3.3 Conclusion for RQ3

The results indicate that:

- ABAC supports dynamic policy adaptation through attribute evaluation.
- ABAC can reduce the need for structural changes to access-control configurations in dynamic environments, while shifting effort toward attribute and policy management.
- RBAC requires manual updates to role assignments and role structures.
- RBAC may be more suitable in stable environments with well-defined access patterns.

These findings align with the literature and provide empirical support for differences in adaptability between the models.

6.4 Overall Synthesis

The results confirm the key claims found in the literature, while also refining them through empirical evidence. ABAC provides higher granularity and flexibility through attribute-based, per-resource evaluation and multi-attribute policy logic. RBAC, by contrast, provides a structured and predictable access model based on role membership.

The findings show that complexity is distributed differently across the models. ABAC concentrates complexity in policy logic and attribute management, while RBAC introduces structural complexity through role design and composition. In dynamic environments, ABAC enables direct adaptation to changes in resource attributes and contextual conditions. RBAC requires updates to role assignments and role structures to reflect such changes. Hybrid approaches appear justified, as ABAC subsumes RBAC and allows roles to be represented as attributes, enabling a combination of RBAC's structured role model with ABAC's flexible policy evaluation (Hu et al., 2014, pp. 5-6; Alsaleh & Kaja, 2024, pp. 19-20; Hamdah & Almoued, 2025, pp. 8-10, 26).

These results support the view that neither RBAC nor ABAC is universally superior. Instead, their suitability depends on the characteristics of the organizational environment.
From a sociotechnical perspective, these differences reflect not only technical model characteristics, but also alternative ways of structuring and governing access control within organizations.

7. Conclusion

This chapter concludes the thesis by summarizing the key findings, answering the research questions and reflecting on the implications of the results. The conclusions are drawn based on the empirical analysis presented in Chapter 6. The chapter is structured by first presenting the main findings of the study in a summary. This is followed by a direct answer to each research question respectively. The chapter then presents overall conclusions, outlines the contributions of the study, discusses its limitations and finally identifies areas for future research.

7.1 Summary of findings

This thesis set out to investigate the differences between Role-based Access Control (RBAC) and Attribute-Based Access Control (ABAC) within the context of a Microsoft Entra-based environment. Through the design and implementation of a controller proof-of-concept (PoC), and the execution of structured validation and scenario-based tests, the study evaluates how these models differ in terms of authorization behavior, implementation complexity and policy maintenance.

The results confirm that both RBAC and ABAC are viable access-control models, but differ significantly in how they express, enforce and manage authorization policies. These differences become particularly evident in scenarios involving context-dependent access conditions, resource attributes and write operations.

7.2 Answer to Research Questions

7.2.1 RQ1 - What are the practical differences between RBAC and ABAC in Microsoft Entra with respect to authorization granularity and flexibility?

The results demonstrate that ABAC generally provides greater granularity and flexibility compared to RBAC.

ABAC evaluates multiple attributes at the time of access and applies these conditions per resource. This allows for fine-grained authorization decisions, resulting in partial visibility of resources and context-dependent outcomes. Users with different attribute configurations observe different subsets of resources even when invoking the same endpoint.

In contrast, RBAC operates through role membership, producing uniform access decisions for users with identical role assignments. Users assigned identical roles receive identical access, regardless of contextual differences such as department, resource state or sensitivity attributes. ABAC also supports conjunctive evaluation of multiple constraints and can express exception-based policies, such as ownership override. RBAC does not natively support this level of expressiveness without introducing additional roles.

The findings indicate that ABAC provides fine-grained, context-aware authorization. Whereas RBAC provides structured, role-based access control with less emphasis on per-request contextual evaluation.

7.2.2 RQ2 - How do RBAC and ABAC differ in terms of implementation complexity and administrative workflow within Microsoft Entra?

The findings show that RBAC and ABAC differ not only in the degree of complexity, but in the nature of that complexity. RBAC is simpler to implement initially, as it relies on predefined roles and straightforward role checks. However, as authorization requirements become more complex, RBAC often requires additional roles to represent combinations of conditions, such as sensitivity and editing permissions. This can lead to increased structural complexity, often referred to as role explosion.
ABAC requires more complex initial implementation, including policy definition, attribute management and evaluation logic. However, this complexity is centralized within the policy model, allowing a single set of rules to handle multiple scenarios without requiring expansion of role structures. The results show that RBAC complexity arises through role proliferation and role combinations while complexity in ABAC arises through policy design and attribute evaluation.

RBAC becomes more complex to manage as authorization requirements grow in complexity. At the same time, RBAC's simplicity and clearly defined structure can make it easier to implement, audit and manage in environments with stable access requirements. ABAC introduces higher initial complexity, but handles multi-dimensional authorization more effectively through centralized policy logic. This illustrates that the difference between the models is not only in complexity level, but in how and where complexity is managed within the system.

7.2.3 RQ3 - How do policy maintenance and change management differ when using RBAC vs ABAC in Entra?

The results indicate that ABAC is more adaptable and easier to maintain in dynamic environments. ABAC policies are evaluated at request time and reflect changes in resource attributes, such as sensitivity or ownership, without requiring modification of role assignments. This allows access control to evolve naturally as system state changes, without requiring changes to role structure, although attribute values must still be managed administratively. RBAC on the other hand, may require updates to role assignments to reflect changes in access requirements. For example, when a resource becomes sensitive, additional roles must be assigned to maintain access. This introduces administrative overhead and delays in reflecting changes.

These findings indicate that ABAC supports dynamic policy adaptation and can reduce the need for structural changes to access-control configurations in dynamic environments, while shifting effort toward attribute and policy management. In contrast, RBAC requires ongoing role management and can become more complex to maintain as authorization requirements evolve.

7.3 Overall Conclusions

The findings of this study confirm the general trends identified in the literature review, while providing empirical validation within a Microsoft Entra-based implementation. RBAC provides simplicity, stability and ease of initial deployment, while ABAC provides flexibility, expressiveness and adaptability. However, neither model is universally optimal. RBAC is well suited for environments characterized by stable roles, clearly defined responsibilities and limited need for contextual access decisions.

ABAC is better suited for environments that require context-aware access control, have dynamic and changing requirements and involve complex relationships between users and resources.

These findings highlight that the suitability of each model depends heavily on the characteristics of the operational environment. This supports the growing consensus in the literature review that hybrid models combining RBAC and ABAC may offer the most effective solution. RBAC can provide a structural foundation, while ABAC refines access decisions through attribute-based policies. These findings are consistent with the literature, particularly Hu et al. (2014), which characterize ABAC as more flexible and context-aware, and Sandhu et al. (1996), which highlights RBAC's structural simplicity and administrative efficiency.

This highlights that access control is not only a technical decision, but also a strategic and organizational choice that shapes how systems are governed and operated.

7.4 Contributions

This thesis contributes to the field by:

- Providing an empirical comparison of RBAC and ABAC within a Microsoft Entra context.
- Demonstrating practical differences in authorization behavior through controlled scenarios.
- Identifying how complexity manifests differently in role-based and attribute-based systems.
- Validating theoretical claims about granularity, flexibility and maintenance through implementation.

In particular, the study contributes empirical evidence to a domain (RBAC and ABAC implementations with Microsoft Entra) that has primarily been explored through theoretical and conceptual research. The findings help bridge the gap between theoretical access-control models and their practical implementation in cloud-based identity systems.

7.5 Limitations

This study has several limitations:

- The evaluation was conducted within a controlled Proof-of-Concept (PoC) environment and may not fully capture the complexity of large-scale enterprise systems.
- The ABAC implementation was applied at the application-layer and does not represent a native Microsoft Entra ABAC system.
- Performance and scalability aspects were not evaluated.
- The selected scenarios emphasize context-dependent and attribute-rich access-control cases, which are commonly associated with ABAC advantages in the literature. As a result, the study focuses on differences in flexibility and expressiveness rather than behavior in static or homogeneous access-control environments.
- The RBAC implementation reflects a simplified, canonical model without extensions or parameterization. While consistent with standard RBAC definitions, more advanced or hybrid RBAC implementations could express additional constraints through expanded role design or application logic.

- The ABAC model assumes that relevant attributes are consistently defined and correctly maintained. In real-world environments, attribute quality and governance may significantly influence authorization behavior and system effectiveness.
- The evaluation was conducted on a limited dataset and user set and may not fully reflect behavior in large-scale deployments.

These limitations indicate that the results should be interpreted as comparative and indicative, rather than absolute.

7.6 Future Work

Future research could extend this study by:

Evaluating hybrid RBAC-ABAC models within Microsoft Entra:
This study identified complementary strengths of RBAC and ABAC, suggesting that hybrid models may offer a practical balance between structural simplicity and contextual flexibility. Future work could investigate how such hybrid approaches can be implemented within Microsoft Entra and how responsibilities should be divided between roles and attribute-based policies.

Investigate scalability and performance impacts of the models:
The current study focused on authorization behavior rather than performance. Future research could analyze how RBAC and ABAC scale in large environments, particularly with respect to request latency, policy evaluation cost and attribute retrieval. This would provide a more comprehensive understanding of trade-offs in real-world deployments.

Investigate attribute governance and policy management:
The effectiveness of ABAC depends heavily on accurate and well-managed attributes. Future research could explore methods for automating attribute management, ensuring data quality and supporting policy lifecycle management. Including validation, auditing and versioning of access-control policies.

Automating policy generation and decision support tools:
Given the increased complexity of ABAC policy design, future work could investigate the use of automated tools for generating, analyzing and optimizing policies.

Conduct an empirical validation in enterprise environments:
Finally, future studies could replicate this research in real-world enterprise settings with larger datasets, more complex organizational structures and operational constraints. This would improve the external validity of the findings and provide insight into practical adoption challenges within Microsoft Entra deployments.

8. Acknowledgements

I would like to start by thanking my Supervisor Peter Bellström, who has provided great feedback and guidance through the thesis writing process.

I would like to thank Sogeti Karlstad, for allowing me to write my thesis at their office and for providing the opportunity to independently shape the direction, methodology and implementation of this thesis within the Microsoft Entra and .NET ecosystem.

I would additionally want to thank Anton Lindqvist for a great first rendition of the thesis Abstract.

Bibliography

Alsaleh, A. S., & Kala, B. F. R. (2024). *Permission-Based Dynamic Access Control Models for Enhanced Data Security*. KTH. https://diva-portal.org/smash/get/diva2:1900749/FULLTEXT01.pdf

Coyne, E. J., Weil, T. R., & Kuhn, R. (2011). *Role engineering: Methods and standards*. IT Professional, 13(6), 54–57. https://doi.org/10.1109/MITP.2011.105

Docker. (n.d). Retrieved 2026-04-02. What is Docker? https://docs.docker.com/get-started/docker-overview/

Glöckler, J., Sedlmeir, J., Frank, M., & Fridgen, G. (2024). *A systematic review of identity and access management requirements in enterprises and potential contributions of self-sovereign identity*. Business & Information Systems Engineering, 66(4), 421-440. https://doi.org/10.1007/s12599-023-00830-x

Hamdah, A. A. N., & Almoued, Y. (2025). *Optimizing Access Control Policies Using Game Theory: A Nash Equilibrium Approach.* University West. https://diva-portal.org/smash/get/diva2:2000182/FULLTEXT01.pdf

Hansen, M. C., Tözün, P., & Hentschel, M. (2026). *Benchmarking role-based access control in data management systems*. In R. Nambiar & M. Poess (Eds.), Performance evaluation and benchmarking: 17th TPC Technology Conference, TPCTC 2025, London, UK, September 1, 2025, revised selected papers (pp. 20–36). Springer. https://doi.org/10.1007/978-3-032-18070-4_2

Hardt, D. (2012). *The OAuth 2.0 authorization framework (RFC 6749)*. Internet Engineering Task Force. https://doi.org/10.17487/RFC6749

Hevner, R, A,. March, T, S,. Park, J,. & Ram, S. (2004). *Design Science in Information Systems Research.* MIS Quarterly Vol. 28 No. 1, pp. 75-105/March 2004. https://www.researchgate.net/publication/201168946_Design_Science_in_Information_Systems_Research

Hu, V., Ferraiolo, D., Kuhn, R., Schnitzer, A., Sandlin, K., Miller, R., & Scarfone, K. (2014). *Guide to Attribute Based Access Control (ABAC) Definition and Considerations*. NIST Special Publication 800-162. https://nvlpubs.nist.gov/nistpubs/specialpublications/nist.sp.800-162.pdf

Lijphart, A. (1971). *Comparative politics and the comparative method*. American Political Science Review, 65(3), 682-693. https://doi.org/10.2307/1955513

Liyanage, D., Pandey, S., Goldstein, J., Cahill, M., Dey, A., Fekete, A., & Röhm, U. (2025). *A benchmark for databases with varying value lengths*. In R. Nambiar & M. Poess (Eds.), Performance evaluation and benchmarking: 17th TPC Technology Conference, TPCTC 2025, London, UK, September 1, 2025, revised selected papers (pp. 1–19). Springer. https://doi.org/10.1007/978-3-032-18070-4.

Microsoft. (2023). Retrieved 2026-05-22. *What is the .NET SDK?* Microsoft Learn. https://learn.microsoft.com/en-us/dotnet/core/sdk

Microsoft. (2024a). Retrieved 2026-04-01. *Implement role-based access control*. Microsoft Learn. https://learn.microsoft.com/en-us/entra/identity-platform/howto-implement-rbac-for-apps

Microsoft. (2024b). Retrieved 2026-05-19. *What are custom security attributes in Microsoft Entra ID?* Microsoft Learn. https://learn.microsoft.com/en-us/entra/fundamentals/custom-security-attributes-overview

Microsoft. (2025a). Retrieved 2026-02-05. *What is Microsoft Entra?* Microsoft Learn. https://learn.microsoft.com/en-us/entra/fundamentals/what-is-entra

Microsoft. (2025b). Retrieved 2026-02-16. *Identity and access management fundamental concepts*. Microsoft Learn. https://learn.microsoft.com/en-us/entra/fundamentals/identity-fundamental-concepts

Microsoft. (2025c). Retrieved 2026-02-16. *What is application management in Microsoft Entra ID?* Microsoft Learn. https://learn.microsoft.com/en-us/entra/identity/enterprise-apps/what-is-application-management?source=recommendations

Microsoft. (2025d). Retrieved 2026-02-16. *What is azure attribute-based access control (Azure ABAC)?* Microsoft Learn. https://learn.microsoft.com/sv-se/azure/role-based-access-control/conditions-overview

Microsoft. (2025e). Retrieved 2026-04-02. *Unit testing best practices for .NET*. Microsoft Learn. https://learn.microsoft.com/en-us/dotnet/core/testing/unit-testing-best-practices

Microsoft. (2025f). Retrieved 2026-05-19. *Overview of role-based access control in Microsoft Entra ID*. Microsoft Learn. https://learn.microsoft.com/en-us/entra/identity/role-based-access-control/custom-overview

Microsoft. (2025g). Retrieved. 2026-05-22. *OAuth 2.0 and OIDC authentication flow in the Microsoft Identity Platform*. Microsoft Learn. https://learn.microsoft.com/nl-nl/entra/identity-platform/v2-protocols

Microsoft. (2026a). Retrieved 2025-05-19. *Microsoft Entra built-in roles*. Microsoft Learn. https://learn.microsoft.com/en-us/entra/identity/role-based-access-control/permissions-reference

Microsoft. (2026b). Retrieved 2026-04-01. *Policy-based authorization in ASP.NET Core*. Microsoft Learn. https://learn.microsoft.com/en-us/aspnet/core/security/authorization/policies?view=aspnetcore-10.0

Microsoft. (2026c). Retrieved 2026-05-17. *What are workload identities? Microsoft Learn*. https://learn.microsoft.com/en-us/entra/workload-id/workload-identities-overview

Microsoft, (n.d.a). Retrieved 2026-04-02. *What is ASP.NET Core?* Microsoft Learn. https://dotnet.microsoft.com/en-us/learn/aspnet/what-is-aspnet-core

Microsoft. (n.d.b). Retrieved 2026-04-02. *What is .NET?* Microsoft Learn. https://dotnet.microsoft.com/en-us/learn/dotnet/what-is-dotnet

Moratore, R., Barbaro, E., & Zhauniarovich, Y. (2025). *IAM Role Diet: A Scalable Approach to Detecting RBAC Data Inefficiencies.* International Conference on Dependable Systems and Networks 55: Proceedings (pp. 126-131). https://doi.org/10.1109/DSN-S65789.2025.00052

Mozilla Developer Network. (2026). Retrieved 2026-05-14. *Overview of HTTP*. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Overview

OAuth.com. (n.d.a). Retrieved. 2026-05-22. *OpenID Connect*. https://www.oauth.com/oauth2-servers/openid-connect/

OAuth.com. (n.d.b). Retrieved 2026-05-22. *ID tokens*. https://www.oauth.com/oauth2-servers/openid-connect/id-tokens/

Pahl, C., Brogi, A., Soldani, J., & Jamshidi, P. (2019). *Cloud container technologies: A state-of-the-art review*. IEEE Transactions on Cloud Computing, 7(3), 677–692. https://doi.org/10.1109/TCC.2017.2702586

Patel, R., & Davidsson, B. (2019). *Forskningsmetodikens grunder: Att planera, genomföra och rapportera en undersökning* (5:e uppl.). Studentlitteratur.

Peffers, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). *A design science research methodology for information systems research*. Journal of Management Information Systems, 24(3), 45–77. https://doi.org/10.2753/MIS0742-1222240302

Ragin, C. C. (2014). *The Comparative Method: Moving Beyond Qualitative and Quantitative Strategies* (1st ed.). University of California Press. http://www.jstor.org/stable/10.1525/j.ctt6wqbwk

Sandhu, R., Coyne, E., Feinstein, H., & Youman, C. (1996). *Role-based access control models*. Computer, 29, no. 2, 38-47.

Sandhu, R. (2001). *Future directions in role-based access control models*. In V. I. Gorodetski, V. A. Skormin, & L. J. Popyack (Eds.), Information assurance in computer networks: Methods, models and architectures for network security (pp. 22–26). Springer. https://doi.org/10.1007/3-540-45116-1_4

Sawyer, S., & Jarrahi, M. H. (2015). *The sociotechnical perspective*. In H. Topi & A. Tucker (Eds.), Information systems and information technology (3rd ed., pp. 1–39). Chapman and Hall/CRC. https://www.researchgate.net/publication/301777084_The_Sociotechnical_Perspective

Sogeti. (2026). Retrieved 2026-03-30. *About us*. https://www.sogeti.com/about-us/

Annex

Appendix A. DSR Guidelines Hevner et al., (2004).

Table A1. Design-Science Research Guidelines, based on Hevner et al., (2004, p. 83).

Guideline	**Description**
Guideline 1: Design as an Artifact	Design-science research should result in a concrete and useful artifact, such as a model, construct, a method, or an instantiation.
Guideline 2: Problem Relevance	The primary goal of design-science research is to create technology-oriented solutions that address relevant business challenges.
Guideline 3: Design Evaluation	The proposed artifact must undergo thorough evaluation regarding its usefulness, effectiveness and quality, using appropriate and systematic and well-executed evaluation methods.
Guideline 4: Research Contributions	Effective design-science research must articulate its contributions in a clear and verifiable manner. Whether it's the foundational knowledge supporting it or the methodologies used during its creation.
Guideline 5: Research Rigor	Both the constructing and assessment of the artifact must rely on rigorous and well-established methodological approaches.
Guideline 6: Design as a Search Process	Developing an effective artifact involves exploring different possible solutions and leveraging available resources. The development needs to adhere to constraints and conditions present in the problem environment.
Guideline 7: Communication of Research	Findings from design-science research should be communicated clearly to both technical audiences and managerial stakeholders.

Appendix B. Activities for conducting DSR Peffers et al., (2007)

Table B1. Activities for conducting DSR, based on Peffers et al., (2007, pp. 52, 55-56).

Activity	**Description**
Activity 1: Problem identification and motivation	Focuses on clearly defining the research problem and explaining why the solving of it matters. To carry out this activity effectively, the researcher needs a solid understanding of the current state of the problem and why a solution is needed.
Activity 2: Define the objectives for a solution.	Involves identifying what a successful solution should achieve. This is based on the defined problem and realistic expectations. Establishing these objectives requires understanding of current challenges and the effectiveness of existing approaches.
Activity 3: Design and Develop	The artifact creation step. This phase includes outlining the artifact's functionality and structure. The design process relies on theoretical knowledge that can guide effective solutions.
Activity 4: Demonstration	The researcher shows how the artifact can address the problem by applying it in scenarios (e.g. experiments, simulations or case studies). Its purpose is to provide initial evidence that the solution works in context.
Activity 5: Evaluation	A comparison about the expected results and actual performance of the artifact. This can involve metrics, qualitative feedback or empirical testing. Based on the findings, researchers decide whether to refine the artifact or proceed.
Activity 6: Communication	The final step is sharing the problem, the artifact and its value to both academic and professional audiences. Researchers typically present their work following structures familiar to their discipline.

Appendix C. Validation Test Results

TestID	Model	Route	ProjectID	User	ConditionTested	AuthzOutcomeExpected	ExpectedVisibleCount	ExpectedReasonForAuthz
V-A1	ABAC	GET:/abac/projects	All	U1	Filtered list access	Pass	1	Sensitive+Inactive+Budget filtering
V-A2	ABAC	GET:/abac/projects	All	U3	Inactive access enabled	Pass	5	Includes inactive projects
V-A3	ABAC	GET:/abac/projects	All	U4	Budget constraint filtering	Pass	1	Budget too low filters projects
V-A4	ABAC	GET:/abac/projects	All	U6	Full attribute access	Pass	6	All attributes satisfied
V-A5	ABAC	GET:/abac/projects/2	P2	U1	Sensitive denied	Denied	0	CanViewSensitive=False
V-A6	ABAC	GET:/abac/projects/2	P2	U2	Sensitive allowed	Pass	1	CanViewSensitive=True
V-A7	ABAC	GET:/abac/projects/5	P5	U2	Department mismatch	Denied	0	Department mismatch
V-A8	ABAC	GET:/abac/projects/3	P3	U2	Inactive denied	Denied	0	CanViewInactiveProjects=False
V-A9	ABAC	GET:/abac/projects/3	P3	U3	Inactive allowed	Pass	1	CanViewInactiveProjects=True
V-A10	ABAC	GET:/abac/projects/4	P4	U4	Budget denied	Denied	0	MaxBudget too low
V-A11	ABAC	GET:/abac/projects/4	P4	U2	Budget allowed	Pass	1	Budget within limit
V-A12	ABAC	GET:/abac/projects/6	P6	U5	Ownership override	Pass	1	Owner override
V-A13	ABAC	GET:/abac/projects/6	P6	U1	Non-owner denied	Denied	0	Fails all attributes
V-A14	ABAC	POST:/abac/projects	N/A	U7	Create allowed	Pass	N/A	AccessLevel=Editor
V-A15	ABAC	POST:/abac/projects	N/A	U1	Create denied	Denied	N/A	AccessLevel!=Editor
V-A16	ABAC	PUT:/abac/projects/1	P1	U7	Update allowed non-sensitive	Pass	N/A	Readable resource
V-A17	ABAC	PUT:/abac/projects/2	P2	U7	Update allowed sensitive	Pass	N/A	CanViewSensitive=True
V-A18	ABAC	PUT:/abac/projects/1	P1	U1	Update denied unreadable	Denied	N/A	Not readable
V-A19	ABAC	PUT:/abac/projects/6	P6	U5	Update via ownership	Pass	N/A	Owner override
V-A20	ABAC	DELETE:/abac/projects/1	P1	U1	Delete denied	Denied	N/A	Not readable
V-A21	ABAC	DELETE:/abac/projects/2	P2	U7	Delete sensitive	Pass	N/A	CanViewSensitive=True
V-A22	ABAC	DELETE:/abac/projects/1	P1	U7	Delete non-sensitive	Pass	N/A	Readable resource
V-A23	ABAC	DELETE:/abac/projects/6	P6	U5	Delete via ownership	Pass	N/A	Owner override
					DB RESET			
V-R1	RBAC	GET:/rbac/projects	All	U1	Baseline viewer access	Pass	2	ProjectViewer role
V-R2	RBAC	GET:/rbac/projects	All	U6	Full access with sensitive role	Pass	6	ProjectViewer+SensitiveViewer
V-R3	RBAC	GET:/rbac/projects	All	U5	Denied no role	Denied	0	Missing ProjectViewer
V-R4	RBAC	GET:/rbac/projects/2	P2	U1	Sensitive denied	Denied	0	Missing SensitiveViewer
V-R5	RBAC	GET:/rbac/projects/2	P2	U6	Sensitive allowed	Pass	1	Has SensitiveViewer
V-R6	RBAC	POST:/rbac/projects	N/A	U7	Create allowed	Pass	N/A	Has ProjectEditor
V-R7	RBAC	POST:/rbac/projects	N/A	U1	Create denied	Denied	N/A	Missing ProjectEditor
V-R8	RBAC	PUT:/rbac/projects/1	P1	U7	Update allowed non-sensitive	Pass	N/A	Has ProjectEditor
V-R9	RBAC	PUT:/rbac/projects/2	P2	U7	Update denied sensitive	Denied	N/A	Missing SensitiveViewer
V-R10	RBAC	PUT:/rbac/projects/2	P2	U8	Update allowed sensitive	Pass	N/A	Has SensitiveViewer
V-R11	RBAC	DELETE:/rbac/projects/1	P1	U7	Delete allowed non-sensitive	Pass	N/A	Has ProjectEditor
V-R12	RBAC	DELETE:/rbac/projects/2	P2	U7	Delete denied sensitive	Denied	N/A	Missing SensitiveViewer
V-R13	RBAC	DELETE:/rbac/projects/2	P2	U8	Delete allowed sensitive	Pass	N/A	Has SensitiveViewer
					DB RESET			

Figure C1. Validation Test: Design of Test Suite.

Id	Model	RequestSignature	ExecutionTimeMs	DecisionCount	ImpactCount	VisibleResourceCount	AccessGranted	PrimaryDenialReason	DenialReasonSummaryJson	GraphApiCalls	Timestamp
1	ABAC	GET:/abac/projects	3847	7	6	1	TRUE	SensitiveDenied	{"SensitiveDenied":4,"InactiveDenied":2,"BudgetTooHigh":2,"DepartmentMismatch":1}	30	2026-05-08T10:52:53.6792420
2	ABAC	GET:/abac/projects	4429	7	2	5	TRUE	DepartmentMismatch	{"DepartmentMismatch":1}	30	2026-05-08T10:53:00.4815718
3	ABAC	GET:/abac/projects	3750	7	6	1	TRUE	BudgetTooHigh	{"BudgetTooHigh":4,"DepartmentMismatch":1}	30	2026-05-08T10:53:06.8941530
4	ABAC	GET:/abac/projects	3291	7	1	6	TRUE			30	2026-05-08T10:53:12.4533550
5	ABAC	GET:/abac/projects/2	878	2	2	0	FALSE	SensitiveDenied	{"SensitiveDenied":1}	5	2026-05-08T10:53:15.8748138
6	ABAC	GET:/abac/projects/2	928	2	1	1	TRUE			5	2026-05-08T10:53:19.2474828
7	ABAC	GET:/abac/projects/5	912	2	2	0	FALSE	DepartmentMismatch	{"DepartmentMismatch":1}	5	2026-05-08T10:53:22.7089500
8	ABAC	GET:/abac/projects/3	1004	2	2	0	FALSE	InactiveDenied	{"InactiveDenied":1}	6	2026-05-08T10:53:26.1702235
9	ABAC	GET:/abac/projects/3	991	2	1	1	TRUE			6	2026-05-08T10:53:29.9824192
10	ABAC	GET:/abac/projects/4	1005	2	2	0	FALSE	BudgetTooHigh	{"BudgetTooHigh":1}	4	2026-05-08T10:53:33.5739593
11	ABAC	GET:/abac/projects/4	891	2	1	1	TRUE			4	2026-05-08T10:53:36.9061769
12	ABAC	GET:/abac/projects/6	426	1	0	1	TRUE			1	2026-05-08T10:53:39.7062281
13	ABAC	GET:/abac/projects/6	910	2	2	0	FALSE	SensitiveDenied	{"SensitiveDenied":1,"InactiveDenied":1,"BudgetTooHigh":1}	6	2026-05-08T10:53:43.1822833
14	ABAC	POST:/abac/projects	561	1	0	1	TRUE			1	2026-05-08T10:53:46.1740771
15	Unknown	POST:/abac/projects	377	1	1	0	FALSE	WriteAccessDenied	{"WriteAccessDenied":1}	1	2026-05-08T10:53:49.1259438
16	ABAC	PUT:/abac/projects/1	896	2	0	2	TRUE			4	2026-05-08T10:53:52.6411264
17	ABAC	PUT:/abac/projects/2	895	2	0	2	TRUE			5	2026-05-08T10:53:55.9874045
18	ABAC	PUT:/abac/projects/1	1129	2	1	1	FALSE	DepartmentMismatch	{"DepartmentMismatch":1,"BudgetTooHigh":1}	4	2026-05-08T10:53:59.4864043
19	ABAC	PUT:/abac/projects/6	457	1	0	1	TRUE			1	2026-05-08T10:54:02.4460751
20	ABAC	DELETE:/abac/projects/1	907	2	1	1	FALSE	DepartmentMismatch	{"DepartmentMismatch":1,"BudgetTooHigh":1}	4	2026-05-08T10:54:05.7537471
21	ABAC	DELETE:/abac/projects/2	903	2	0	2	TRUE			5	2026-05-08T10:54:09.0026169
22	ABAC	DELETE:/abac/projects/1	751	2	1	1	FALSE	DepartmentMismatch	{"DepartmentMismatch":1,"BudgetTooHigh":1}	4	2026-05-08T10:54:12.0266261
23	ABAC	DELETE:/abac/projects/6	427	1	0	1	TRUE			1	2026-05-08T10:54:15.4618199

Summary
totalCount:23
minExecutionTimeMs:377
maxExecutionTimeMs:4429
avgExecutionTimeMs:1328.91
accessGrantedCount:14
accessDeniedCount:9
totalGraphApiCalls:192

Figure C2. Validation Test: ABAC-Results.

Id	Model	RequestSignature	ExecutionTimeMs	DecisionCount	ImpactCount	VisibleResourceCount	AccessGranted	PrimaryDenialReason	DenialReasonSummaryJson	GraphApiCalls	Timestamp
1	RBAC	GET:/rbac/projects	12	1	1	2	TRUE	MissingSensitiveViewerRole	{"MissingSensitiveViewerRole":1}	0	2026-05-08T10:54:26.4602487
2	RBAC	GET:/rbac/projects	2	1	0	6	TRUE			0	2026-05-08T10:54:28.6130962
3	Unknown	GET:/rbac/projects	0	1	1	0	FALSE	MissingRole	{"MissingRole":1}	0	2026-05-08T10:54:30.9072578
4	RBAC	GET:/rbac/projects/2	3	2	1	1	FALSE	MissingSensitiveViewerRole	{"MissingSensitiveViewerRole":1}	0	2026-05-08T10:54:33.1919756
5	RBAC	GET:/rbac/projects/2	1	2	0	1	TRUE			0	2026-05-08T10:54:35.3180239
6	RBAC	POST:/rbac/projects	13	1	0	1	TRUE			0	2026-05-08T10:54:37.6517620
7	Unknown	POST:/rbac/projects	0	1	1	0	FALSE	MissingRole	{"MissingRole":1}	0	2026-05-08T10:54:40.0563116
8	RBAC	PUT:/rbac/projects/1	8	2	0	2	TRUE			0	2026-05-08T10:54:42.5410208
9	RBAC	PUT:/rbac/projects/2	1	2	1	1	FALSE	MissingSensitiveViewerRole	{"MissingSensitiveViewerRole":1}	0	2026-05-08T10:54:45.1875672
10	RBAC	PUT:/rbac/projects/2	3	2	0	2	TRUE			0	2026-05-08T10:54:47.5546916
11	RBAC	DELETE:/rbac/projects/1	6	2	0	2	TRUE			0	2026-05-08T10:54:49.9277105
12	RBAC	DELETE:/rbac/projects/2	11	2	0	2	TRUE			0	2026-05-08T10:54:52.2129460
13	RBAC	DELETE:/rbac/projects/2	9	1	0	1	FALSE			0	2026-05-08T10:54:55.7637863

Summary
totalCount:13
minExecutionTimeMs:0
maxExecutionTimeMs:13
avgExecutionTimeMs:5.31
accessGrantedCount:8
accessDeniedCount:5
totalGraphApiCalls:0

Figure C3. Validation Test: RBAC-Results

Appendix D. Scenario Test Results

Test	User	Route	Purpose	Expected results
S1-T1	U1	GET /abac/projects	Strong filtering	VisibleResourceCount = 1, multiple denial reasons (Sensitive, Inactive, Budget)
S1-T2	U1	GET /rbac/projects	Baseline visibility	VisibleResourceCount = 2, non-sensitive only
S1-T3	U3	GET /abac/projects	Inactive inclusion	VisibleResourceCount = 5, inactive projects included
S1-T4	U3	GET /rbac/projects	Role unchanged	VisibleResourceCount = 2, identical to U1
S1-T5	U4	GET /abac/projects	Budget filtering	VisibleResourceCount = 1, high-budget projects excluded
S1-T6	U4	GET /rbac/projects	Budget ignored	VisibleResourceCount = 2, same as U1 and U3
S1-T7	U6	GET /abac/projects	Full ABAC access	VisibleResourceCount = 6, all constraints satisfied
S1-T8	U6	GET /rbac/projects	Full RBAC access	VisibleResourceCount = 6, SensitiveViewer role present

Figure D1. Scenario1: Design of Test Suite.

Id	Model	RequestSignature	ExecutionTimeMs	DecisionCount	ImpactCount	VisibleResourceCount	AccessGranted	PrimaryDenialReason	DenialReasonSummaryJson	GraphApiCalls	Timestamp
1	ABAC	GET:/abac/projects	3717	7	6	1	TRUE	SensitiveDenied	{"SensitiveDenied":4,"InactiveDenied":2,"BudgetTooHigh":2,"DepartmentMismatch":1}	30	2026-05-08T11:05:12.9781611
2	RBAC	GET:/rbac/projects	83	1	1	2	TRUE	MissingSensitiveViewerRole	{"MissingSensitiveViewerRole":1}	0	2026-05-08T11:05:15.5182058
3	ABAC	GET:/abac/projects	3715	7	2	5	TRUE	DepartmentMismatch	{"DepartmentMismatch":1}	30	2026-05-08T11:05:21.5348779
4	RBAC	GET:/rbac/projects	5	1	1	2	TRUE	MissingSensitiveViewerRole	{"MissingSensitiveViewerRole":1}	0	2026-05-08T11:05:24.0605819
5	ABAC	GET:/abac/projects	3338	7	6	1	TRUE	BudgetTooHigh	{"BudgetTooHigh":4,"DepartmentMismatch":1}	30	2026-05-08T11:05:29.7255299
6	RBAC	GET:/rbac/projects	1	1	1	2	TRUE	MissingSensitiveViewerRole	{"MissingSensitiveViewerRole":1}	0	2026-05-08T11:05:32.1029199
7	ABAC	GET:/abac/projects	3150	7	1	6	TRUE			30	2026-05-08T11:05:37.6469380
8	RBAC	GET:/rbac/projects	1	1	0	6	TRUE			0	2026-05-08T11:05:40.7260506

Summary
totalCount:8
minExecutionTimeMs:1
maxExecutionTimeMs:3717
avgExecutionTimeMs:1751.25
accessGrantedCount:8
accessDeniedCount:0
totalGraphApiCalls:120

Figure D2. Scenario1: Results.

Test	User	Endpoint	Purpose	Expected result
S2-T1	U1	GET /abac/projects/2	Verify ABAC denies sensitive access without	Denied
S2-T2	U1	GET /rbac/projects/2	Verify RBAC denies sensitive access without	Denied
S2-T3	U2	GET /abac/projects/2	Demonstrate attribute-based permission for s	Allowed
S2-T4	U2	GET /rbac/projects/2	Demonstrate RBAC restriction without Sensiti	Denied
S2-T5	U6	GET /abac/projects/2	Validate full attribute-based access	Allowed
S2-T6	U6	GET /rbac/projects/2	Validate full role-based access	Allowed
S2-T7	U2	GET /abac/projects/5	ute evaluation (department mismatch override	Denied

Figure D3. Scenario2: Design of Test Suite.

Id	Model	RequestSignature	ExecutionTimeMs	DecisionCount	ImpactCount	VisibleResourceCount	AccessGranted	PrimaryDenialReason	DenialReasonSummaryJson	GraphApiCalls	Timestamp
1	ABAC	GET:/abac/projects/2	1084	2	2	0	FALSE	SensitiveDenied	{"SensitiveDenied":1}	5	2026-05-08T11:26:27.8144318
2	RBAC	GET:/rbac/projects/2	1	2	1	1	FALSE	MissingSensitiveViewerRole	{"MissingSensitiveViewerRole":1}	0	2026-05-08T11:26:30.2946051
3	ABAC	GET:/abac/projects/2	964	2	1	1	TRUE			5	2026-05-08T11:26:33.8229666
4	RBAC	GET:/rbac/projects/2	1	2	1	1	FALSE	MissingSensitiveViewerRole	{"MissingSensitiveViewerRole":1}	0	2026-05-08T11:26:36.1947446
5	ABAC	GET:/abac/projects/2	1060	2	1	1	TRUE			5	2026-05-08T11:26:39.6480413
6	RBAC	GET:/rbac/projects/2	1	2	0	1	TRUE			0	2026-05-08T11:26:42.2465578
7	ABAC	GET:/abac/projects/5	887	2	2	0	FALSE	DepartmentMismatch	{"DepartmentMismatch":1}	5	2026-05-08T11:26:45.5311138

Summary
totalCount:7
minExecutionTimeMs:1
maxExecutionTimeMs:1084
avgExecutionTimeMs:571.14
accessGrantedCount:3
accessDeniedCount:4
totalGraphApiCalls:20

Figure D4. Scenario2: Results.

Test	User	Endpoint	Purpose	Expected result
S3-T1	U5	GET /abac/projects/6	Demonstrate ownership override in ABAC (access granted despite failing all attribute constraints)	Allowed
S3-T1	U5	GET /rbac/projects/6	Demonstrate that RBAC cannot express ownership-based access without roles	Denied
S3-T3	U1	GET /abac/projects/6	Validate that ABAC denies access when no ownership and all attribute constraints fail	Denied
S3-T4	U1	GET /rbac/projects/6	Validate baseline RBAC denial when required roles are missing	Denied

Figure D5. Scenario3: Design of Test Suite.

Id	Model	RequestSignature	ExecutionTimeMs	DecisionCount	ImpactCount	VisibleResourceCount	AccessGranted	PrimaryDenialReason	DenialReasonSummaryJson	GraphApiCalls	Timestamp
1	ABAC	GET:/abac/projects/6	545	1	0	1	TRUE			1	2026-05-08T11:35:27.9156717
2	Unknown	GET:/rbac/projects/6	0	1	1	0	FALSE	MissingRole	{"MissingRole":1}	0	2026-05-08T11:35:30.1547073
3	ABAC	GET:/abac/projects/6	883	2	2	0	FALSE	SensitiveDenied	{"SensitiveDenied":1,"InactiveDenied":1,"BudgetTooHigh":1}	6	2026-05-08T11:35:34.0487019
4	RBAC	GET:/rbac/projects/6	1	2	1	1	FALSE	MissingSensitiveViewerRole	{"MissingSensitiveViewerRole":1}	0	2026-05-08T11:35:36.3169429

Summary
totalCount:4
minExecutionTimeMs:0
maxExecutionTimeMs:883
avgExecutionTimeMs:357.25
accessGrantedCount:1
accessDeniedCount:3
totalGraphApiCalls:7

Figure D6. Scenario3: Results.

Test	User	Endpoint	Expected Result	Purpose
S4-T1	U7	PUT /abac/projects/1	Allowed	Baseline write capability under ABAC
S4-T2	U7	PUT /rbac/projects/1	Denied	RBAC requires SensitiveViewer when resource becomes sensitive
S4-T3	U1	PUT /abac/projects/1	Denied	Demonstrates requirement for write privileges and attribute satisfaction
S4-T4	U1	PUT /rbac/projects/1	Denied	Demonstrates requirement for ProjectEditor role
S4-T5	U7	PUT /abac/projects/2	Allowed	ABAC permits sensitive write based on attributes (CanViewSensitive)
S4-T6	U7	PUT /rbac/projects/2	Denied	RBAC denies sensitive writes without SensitiveViewer role
S4-T7	U8	PUT /abac/projects/2	Allowed	Full attribute satisfaction allows write
S4-T8	U8	PUT /rbac/projects/2	Allowed	Combined roles enable write (ProjectEditor + SensitiveViewer)
			DB RESET	
S4-T9	U7	POST /abac/projects	Allowed	Attribute-based write permission (AccessLevel = Editor)
S4-T10	U7	POST /rbac/projects	Allowed	Role-based write permission (ProjectEditor)
S4-T11	U7	DELETE /rbac/projects/2	Denied	RBAC denies sensitive delete without SensitiveViewer role
S4-T12	U7	DELETE /abac/projects/2	Allowed	ABAC allows based on attribute evaluation

Figure D7. Scenario4: Design of Test Suite.

Id	Model	RequestSignature	ExecutionTimeMs	DecisionCount	ImpactCount	VisibleResourceCount	AccessGranted	PrimaryDenialReason	DenialReasonSummaryJson	GraphApiCalls	Timestamp
1	ABAC	PUT:/abac/projects/1	1140	2	0	2	TRUE			4	2026-05-08T11:56:48.4510827
2	RBAC	PUT:/rbac/projects/1	5	2	1	1	FALSE	MissingSensitiveViewerRole	{"MissingSensitiveViewerRole":1}	0	2026-05-08T11:56:50.8335418
3	ABAC	PUT:/abac/projects/1	855	2	1	1	FALSE	SensitiveDenied	{"SensitiveDenied":1}	5	2026-05-08T11:56:54.0896821
4	Unknown	PUT:/rbac/projects/1	0	1	1	0	FALSE	MissingRole	{"MissingRole":1}	0	2026-05-08T11:56:56.5205973
5	ABAC	PUT:/abac/projects/2	951	2	0	2	TRUE			5	2026-05-08T11:56:59.7709266
6	RBAC	PUT:/rbac/projects/2	1	2	1	1	FALSE	MissingSensitiveViewerRole	{"MissingSensitiveViewerRole":1}	0	2026-05-08T11:57:02.1094520
7	ABAC	PUT:/abac/projects/2	879	2	0	2	TRUE			5	2026-05-08T11:57:05.7307924
8	RBAC	PUT:/rbac/projects/2	1	2	0	2	TRUE			0	2026-05-08T11:57:08.1852886

Summary
totalCount:8
minExecutionTimeMs:0
maxExecutionTimeMs:1140
avgExecutionTimeMs:479
accessGrantedCount:4
accessDeniedCount:4
totalGraphApiCalls:19

Id	Model	RequestSignature	ExecutionTimeMs	DecisionCount	ImpactCount	VisibleResourceCount	AccessGranted	PrimaryDenialReason	DenialReasonSummaryJson	GraphApiCalls	Timestamp
1	ABAC	POST:/abac/projects	512	1	0	1	TRUE			1	2026-05-08T11:57:19.1418875
2	RBAC	POST:/rbac/projects	6	1	0	1	TRUE			0	2026-05-08T11:57:21.2748051
3	RBAC	DELETE:/rbac/projects/2	4	2	1	1	FALSE	MissingSensitiveViewerRole	{"MissingSensitiveViewerRole":1}	0	2026-05-08T11:57:23.6484644
4	ABAC	DELETE:/abac/projects/2	963	2	0	2	TRUE			5	2026-05-08T11:57:27.1462200

Summary
totalCount:4
minExecutionTimeMs:4
maxExecutionTimeMs:963
avgExecutionTimeMs:371.25
accessGrantedCount:3
accessDeniedCount:1
totalGraphApiCalls:6

Figure D8. Scenario4: Results.

Appendix E. Implementation Log

This appendix uses simplified numbered references ([1-19]) to document implementation-related sources.

Development Approach

The Proof-of-Concept (PoC) was developed iteratively following Design Science Research principles. The artefact was progressively refined to enable a controlled comparison between RBAC and ABAC within Microsoft Entra.
The implementation approach was guided by established Microsoft documentation and architectural best practices for ASP.NET Core applications. This ensured that design decisions were grounded in accepted industry standards, rather than ad hoc implementation choices.
This appendix focuses on the final architecture and retains design decisions, including how these were informed and guided by technical sources.

System Architecture Overview

The PoC artefact was implemented as an ASP.NET Core 8 Web API, structured as a layered system consisting of:

- HTTP request pipeline (middleware and authorization)
- Controllers (endpoint definitions)
- Authorization layer (RBAC and ABAC implementations)
- Services (measurement logging and attribute retrieval through Microsoft Graph API calls)
- Persistence layer (Entity Framework Core with SQLite database)

The request flow follows the ASP.NET Core pipeline:
MeasurementMiddleware → Authentication → Authorization → Controllers → Data Layer

This architectural structure is aligned with ASP.NET Core's middleware-based request processing model, where requests are handled through a sequential pipeline of middleware components [1]. Each component can process the request, perform operations before and after invoking the next component and control whether the request continues through the pipeline [1].

Backend Architecture and Routing

The PoC artefact uses a controller-based architecture following Microsoft's ASP.NET Core Web API design guidelines [2][3]. Controllers are responsible for handling HTTP requests and defining API endpoints, providing a structured and explicit interface to the system.

This decision was made to maintain clear endpoint definitions and support dependency injection of services and persistence layers. It also allowed the authorization models to be implemented in parallel (each using a separate controller), which simplified the controlled comparison between the models and avoided internal dependencies between them.

The system exposes two parallel endpoint structures:

- /rbac/ …
- /abac/ …

These endpoints implement identical functionality but differ only in authorization logic.

This decision ensured that the authorization is the only experimental variable, which was essential for a valid comparison between the models.

Persistence Layer and Data Management

The system uses a SQLite database with Entity Framework Core (EF Core), following Microsoft's guidance for ASP.NET Core data access [4][5][6].

Entity Framework Core was selected as it provides object-relational mapping (ORM) capabilities, enabling developers to work with databases using .NET objects and reducing the need for manual data access code [6]. It also supports asynchronous database operations, allowing efficient handling of I/O-bound tasks [5].

In addition, EF Core integrates with ASP.NET Core's dependency injection system, where the DbContext is typically registered with a scoped lifetime [4]. This means that a new DbContext instance is created for each HTTP request, aligning with the unit-of-work pattern and ensuring that each request operates in isolation [4].

This design prevents shared mutable state between requests and ensures safe, independent database interactions. The database is initialized using deterministic data seeding, enabling each automated test session to start from a consistent and reproducible state. This approach follows Microsoft's recommended seeding patterns in EF Core [7].

Authentication Architecture (Microsoft Entra)

Authentication is implemented using Microsoft Entra ID with JWT Bearer tokens, following Microsoft's identity platform documentation [9][10]. The system validates access tokens using ASP.NET Core authentication middleware, extracts relevant claims (such as roles and object identifiers) and constructs a claims-based identity used for authorization decisions [9][10].
For automated testing, the OAuth 2.0 Resource Owner Password Credentials (ROPC) flow was used [11]. This design choice enables automated and repeatable token acquisition without requiring interactive user authentication.
Although ROPC is not recommended for production use due to security limitations, Microsoft's documentation states that it should only be used when more secure authentication flows are not viable and in scenarios requiring a high degree of trust, such as controlled or non-interactive environments [11].

Authorization Architecture

RBAC Implementation

RBAC was implemented using Microsoft Entra "application roles" [12].
The system defines the roles "ProjectViewer", "SensitiveViewer" and "ProjectEditor". These roles are included as role claims in issued tokens and used to enforce authorization logic within the application [12].
Authorization is enforced using ASP.NET Core's built-in role-based mechanisms, specifically the [Authorize(Roles="...")] attribute [13]. This approach follows Microsoft's recommended RBAC implementation pattern, where roles are mapped to permissions and evaluated during request processing [12].

The RBAC design evaluates access at the request-level and uses additive role composition, allowing users to accumulate permissions based on assigned roles while avoiding attribute-based logic.

ABAC Implementation

ABAC was implemented using ASP.NET Core policy-based authorization [14]. This model relies on custom authorization requirements, authorization handlers and resource-level evaluation to determine access decisions.
User attributes are stored in Microsoft Entra using "Custom Security Attributes" [15].
These attributes are retrieved dynamically using Microsoft Graph API for each request and are used as input for authorization decisions.

This design aligns with attribute-based authorization principles, where access is evaluated at runtime based on multiple attributes, enabling fine-grained and context-aware access control [14][15].

Measurement and Instrumentation Design

A key requirement of the thesis was to measure authorization behavior without affecting the implementation logic. To achieve this, a "MeasurementService" was implemented using ASP.NET Core Middleware. This design follows Microsoft's guidance on handling cross-cutting concerns such as logging, monitoring and instrumentation [1][16].
Middleware components operate as part of the request processing pipeline and can perform logic both before and after passing control to the next component, enabling full coverage of the request lifecycle [1]. In contrast, filters execute only within the controller/action pipeline, making them less suitable for system-wide instrumentation [16].
The middleware-based approach was therefore selected because it operates independently of controllers, ensures consistent measurements across all requests, and captures behavior throughout the entire request lifecycle.

This design also aligns with observability principles, where telemetry and instrumentation should be implemented in a non-invasive manner, allowing the system to be monitored without affecting its core functionality [17].

Measurement Model Design

The system records structured data per request, including "decisionCount, visibleResourceCount, accessGranted", as well as denial reasons. For a full rundown of the measurement variables see (3.3.4 *MeasurementService,* Table 4).
This measurement framework was explicitly designed to reflect observable authorization behavior and support a direct comparison between RBAC and ABAC.

Reproducibility and Experimental Control

To ensure experimental validity and reproducibility, the system was designed with strict control over execution conditions:

- Deterministic dataset seeding is applied to ensure a consistent initial state.
- The database can be reset between runs using a dedicated DELETE endpoint (/dev/reset-db).
- API calls are executed via Bash scripts.
- Identical endpoints are used across both RBAC and ABAC models.

Scripted execution is performed using command-line tools such as cURL, which allow HTTP requests to be executed programmatically and consistently across test runs [18][19]. This approach ensures that all requests are identical across experiments, eliminating variability introduced by manual interaction.

Together, these measures ensure that all comparisons are performed under controlled and reproducible conditions. This allows observed differences to be attributed solely to the authorization model under evaluation.

References Appendix E

[1] ASP.NET Core Middleware - https://learn.microsoft.com/en-us/aspnet/core/fundamentals/middleware/?view=aspnetcore-10.0
[2] ASP.NET Core Web API controllers - https://learn.microsoft.com/en-us/aspnet/core/web-api/
[3] Build a Web API with ASP.NET Core (training module) - https://learn.microsoft.com/en-us/training/modules/build-web-api-aspnet-core/1-introduction
[4] DbContext lifetime and configuration - https://learn.microsoft.com/en-us/ef/core/dbcontext-configuration/
[5] EF Core saving data - https://learn.microsoft.com/en-us/ef/core/saving/basic
[6] Entity Framework Core overview - https://learn.microsoft.com/en-us/ef/core/

[7] EF Core data seeding - https://learn.microsoft.com/en-us/ef/core/modeling/data-seeding
[8] Applying EF Core migrations - https://learn.microsoft.com/en-us/ef/core/managing-schemas/migrations/applying
[9] Build and secure an ASP.NET Core Web API with Microsoft identity platform - https://learn.microsoft.com/en-us/entra/identity-platform/tutorial-web-api-dotnet-core-build-app
[10] Configure JWT bearer authentication in ASP.NET Core - https://learn.microsoft.com/en-us/aspnet/core/security/authentication/configure-jwt-bearer-authentication
[11] OAuth 2.0 Resource Owner Password Credentials (ROPC) - https://learn.microsoft.com/en-us/entra/identity-platform/v2-oauth-ropc
[12] Add app roles in Microsoft Entra ID - https://learn.microsoft.com/en-us/entra/identity-platform/howto-add-app-roles-in-apps
[13] Role-based authorization in ASP.NET Core - https://learn.microsoft.com/en-us/aspnet/core/security/authorization/roles
[14] Policy-based authorization in ASP.NET Core - https://learn.microsoft.com/en-us/aspnet/core/security/authorization/policies
[15] Custom security attributes in Microsoft Entra ID - https://learn.microsoft.com/en-us/entra/fundamentals/custom-security-attributes-overview
[16] ASP.NET Core filters - https://learn.microsoft.com/en-us/aspnet/core/mvc/controllers/filters
[17] Observability in .NET (OpenTelemetry) - https://learn.microsoft.com/en-us/dotnet/core/diagnostics/observability-with-otel
[18] Call an ASP.NET Core Web API with cURL - https://learn.microsoft.com/en-us/entra/identity-platform/howto-call-a-web-api-with-curl
[19] cURL documentation - https://curl.se/docs/manpage.html